

Apostila Front-End 2

JavaScript na prática com o App Livros

Curso construído aula a aula: dos fundamentos da linguagem (variáveis, condicionais, laços, funções e eventos) até uma SPA completa com roteamento por hash, componentes, assincronismo e consumo de API, fechando com o uso de LLMs no fluxo de trabalho do desenvolvedor.

Sumário

1. Capítulo 1: Introdução ao projeto e preparação do ambiente
2. Capítulo 2: Variáveis e tipos de dados
3. Capítulo 3: Tomando decisões com if, else e switch
4. Capítulo 4: Repetindo tarefas com for e while
5. Capítulo 5: Arrays e objetos, organizando os dados da aplicação
6. Capítulo 6: Funções, com retorno e sem retorno
7. Capítulo 7: DOM e eventos de escuta
8. Capítulo 8: ES Modules e componentização
9. Capítulo 9: SPA e roteamento por hash
10. Capítulo 10: Assincronismo, o tempo no JavaScript
11. Capítulo 11: Consumindo APIs com fetch
12. Capítulo 12: Estratégias de cache e os armazenamentos do navegador
13. Capítulo 13: Programando com IA, engenharia de prompt para desenvolvedores
14. Capítulo 14: Fechando o ciclo, o projeto completo e os próximos passos

Capítulo 1: Introdução ao projeto e preparação do ambiente

O que vamos construir neste curso

Antes de escrever a primeira linha de código, precisamos combinar o destino da viagem. Ao longo das aulas de sábado nós vamos construir, passo a passo, uma aplicação web completa chamada App Livros. Ela é uma SPA, sigla em inglês para Single Page Application, ou seja, uma aplicação de página única.

O que isso significa na prática? Significa que o navegador carrega um único arquivo HTML e, a partir dele, todo o conteúdo das páginas (Home, Sobre, Serviços, Contato, Cadastro) é montado e trocado pelo JavaScript, sem recarregar a página inteira. É exatamente assim que funcionam aplicações modernas como Gmail e Twitter, e é assim que frameworks como React, Vue e Angular trabalham por baixo dos panos.

A diferença é que nós vamos fazer tudo sem framework nenhum, usando apenas JavaScript puro (o famoso Vanilla JS). Quando você entender como uma SPA funciona por dentro, aprender qualquer framework depois vira uma questão de sintaxe, porque os conceitos você já domina.

Por que começar pelos fundamentos

Cada aula adiciona uma peça no projeto, e cada peça depende de um fundamento da linguagem:

1. Variáveis e tipos: guardar informações (Capítulo 2)
2. Condicionais if, else e switch: tomar decisões (Capítulo 3)
3. Laços for e while: repetir tarefas (Capítulo 4)
4. Arrays e objetos: organizar dados, como a lista de rotas do menu (Capítulo 5)
5. Funções: empacotar comportamentos, como as funções que geram páginas (Capítulo 6)
6. DOM e eventos de escuta: reagir ao usuário, como o envio do formulário (Capítulo 7)
7. Módulos e componentes: organizar o projeto em arquivos (Capítulo 8)
8. Roteamento por hash: navegar entre páginas sem recarregar (Capítulo 9)
9. Assincronismo: entender o tempo no JavaScript (Capítulo 10)
10. Fetch e APIs: buscar dados externos, como o CEP no ViaCEP (Capítulo 11)

Repare que a ordem não é aleatória. É a ordem em que o próprio projeto foi crescendo nos commits do repositório. Se você olhar o histórico do Git vai ver essa evolução registrada aula a aula.

A estrutura do projeto

Esta é a organização de pastas que vamos construir e manter até o final:

```

app_livros/
|-- index.html           página única da SPA
|-- src/
|   |-- css/
|   |   |-- microframework.css  estilos com metodologia BEM
|   |-- img/                  imagens usadas nos cards
|   |-- js/
|       |-- main.js           ponto de entrada: roteador e render
|       |-- components/
|           |-- navbar/       menu dinâmico
|           |-- footer/
|           |-- rotas/         definição central das rotas
|           |-- paginas/      home, sobre, servicos, contato, formCad

```

O ponto de partida: index.html

Todo o HTML da aplicação se resume a isto:

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>APP livros</title>
  <link rel="stylesheet" href="src/css/microframework.css">
</head>
<body>
  <header id="navbar"></header>
  <main id="app"></main>
  <script src="src/js/main.js" type="module"></script>
</body>
</html>

```

Guarde bem esses dois elementos com id, porque eles são os pontos de montagem da aplicação:

- `<header id="navbar">` é onde o JavaScript vai desenhar o menu
- `<main id="app">` é onde o JavaScript vai desenhar o conteúdo de cada página

Repare também no atributo `type="module"` do script. Ele avisa o navegador que vamos usar ES Modules, o sistema de importação e exportação de arquivos do JavaScript moderno, que estudaremos no Capítulo 8.

Como executar o projeto

Por causa do `type="module"`, o navegador exige que o projeto seja servido por um servidor HTTP. Abrir o arquivo direto com dois cliques (protocolo `file://`) não funciona.

Opção 1, a mais usada em aula: extensão Live Server no VS Code. Clique com o botão direito no `index.html` e escolha Open with Live Server.

Opção 2, pelo terminal:

```
# com Python instalado
python -m http.server 8000

# ou com Node instalado
npx serve
```

Depois acesse `http://localhost:8000` (ou a porta indicada) no navegador.

Ferramenta essencial: o console do navegador

Aperte F12 no navegador e abra a aba Console. Esse vai ser nosso melhor amigo durante o curso inteiro. É nele que:

- aparecem os erros do nosso código, com o arquivo e a linha do problema
- imprimimos valores com `console.log()` para investigar o que está acontecendo
- testamos pequenos trechos de código antes de colocar no projeto

Acostume-se a programar com o console sempre aberto. Grande parte de aprender a programar é aprender a ler mensagens de erro sem medo.

Resumo do capítulo

- Vamos construir uma SPA em JavaScript puro, sem frameworks
- O HTML tem uma única página com dois pontos de montagem: `#navbar` e `#app`
- Todo o resto da interface é gerado por JavaScript
- O projeto precisa rodar em um servidor HTTP por usar ES Modules
- O console do navegador (F12) é a principal ferramenta de estudo e depuração

Para praticar

1. Crie a estrutura de pastas do projeto na sua máquina.
2. Crie o `index.html` com os dois pontos de montagem.
3. Rode o projeto com o Live Server e confirme que a página abre sem erros no console.
4. Escreva `console.log("Olá, curso de Front-End 2")` em um arquivo `main.js` e confirme que a mensagem aparece no console.

Referências

- MDN Web Docs, Introdução ao JavaScript: https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/First_steps
- MDN Web Docs, SPA (Single Page Application): <https://developer.mozilla.org/pt-BR/docs/Glossary/SPA>
- W3Schools, JavaScript Tutorial: <https://www.w3schools.com/js/>

Capítulo 2: Variáveis e tipos de dados

Guardando informações

Todo programa precisa guardar informações para trabalhar com elas depois: o nome de um usuário, o endereço da rota atual, a lista de serviços. Em JavaScript, guardamos informações em variáveis.

Pense na variável como uma caixa etiquetada. A etiqueta é o nome da variável e o conteúdo é o valor. Quando escrevemos:

```
let hash = window.location.hash;
```

estamos criando uma caixa chamada `hash` e guardando dentro dela o endereço atual da página. Essa linha existe de verdade no nosso `main.js` e é uma das mais importantes do projeto, como veremos no capítulo de roteamento.

As três formas de declarar: var, let e const

O JavaScript tem três palavras reservadas para criar variáveis. Na prática do nosso curso usamos duas delas.

const

Use `const` quando o valor não vai ser reatribuído. É a nossa escolha padrão no projeto:

```
const app = document.getElementById('app');
const mapaDeRotas = {};
const rota404 = { pagina: () => `<div> Página não encontrada 404 </div>` };
```

O elemento `app` é capturado uma vez e nunca trocamos a referência, então `const` comunica essa intenção para quem lê o código.

Atenção a um detalhe importante: `const` impede a reatribuição, mas não congela o conteúdo de objetos e arrays. Repare que declaramos `const mapaDeRotas = {}` e depois adicionamos rotas dentro dele sem problema nenhum. O que não pode é fazer `mapaDeRotas = outraCoisa`.

let

Use `let` quando o valor precisa mudar ao longo do programa:

```
let hash = window.location.hash || '#inicio';
// mais tarde, quando o usuário navega:
hash = window.location.hash;
```

O `hash` muda toda vez que o usuário clica em um link do menu, por isso ele é `let` no nosso roteador.

var

O `var` é a forma antiga, de antes de 2015. Ele ainda funciona, mas tem comportamentos confusos de escopo que causam bugs difíceis de achar. Regra da nossa sala de aula: não usamos `var` em código novo. Se aparecer em algum material antigo na internet, você sabe que é código legado.

Tipos de dados primitivos

O JavaScript é uma linguagem de tipagem dinâmica: a variável assume o tipo do valor que recebe. Os tipos que mais usamos no projeto:

```
// string: textos, sempre entre aspas
const titulo = "Jogo das quartas de final da copa do mundo de 2002";

// number: números inteiros ou decimais, sem aspas
let contador = 0;
const preco = 49.90;

// boolean: verdadeiro ou falso
const menuAberto = false;

// undefined: a caixa existe mas ninguém colocou nada dentro
let mensagem;
console.log(mensagem); // undefined

// null: ausência intencional de valor
let usuarioLogado = null;
```

Você pode descobrir o tipo de qualquer valor com o operador `typeof`:

```
console.log(typeof titulo);    // "string"
console.log(typeof contador);  // "number"
console.log(typeof menuAberto); // "boolean"
```

Template strings: a ferramenta que usamos o curso inteiro

Existem três formas de escrever strings: aspas simples, aspas duplas e crase (acento grave). A crase cria uma template string, e ela é tão importante que praticamente toda página do nosso projeto é construída com ela.

A template string tem dois superpoderes:

1. Aceita quebras de linha, o que permite escrever HTML de vários parágrafos dentro do JavaScript
2. Aceita interpolação de valores com a sintaxe `${expressao}`

Veja como a página Sobre do projeto usa os dois recursos:

```
function sobre(app){
  const sobre = `

# 


```

E veja a interpolação em ação na lista de contatos:

```
li.textContent = `O Assunto é ${assunto}
e o email é ${email}
e a mensagem é ${mensagem}`;
```

Tudo o que estiver dentro de `${}` é avaliado como JavaScript e o resultado entra no texto. É assim que misturamos dados com HTML durante todo o curso.

Nomeando bem as variáveis

Regras técnicas: o nome pode conter letras, números, `_` e `$`, mas não pode começar com número nem conter espaços ou acentos.

Regras de qualidade, que valem mais que as técnicas:

- Use nomes que descrevem o conteúdo: `mapaDeRotas` e não `m`
- Use o padrão `camelCase`, como em `telaCadastro` e `capturarFormulario`
- Prefira nomes em um idioma só no projeto inteiro

Um código bem nomeado quase dispensa comentários. Compare `const x = document.getElementById('app')` com `const app = document.getElementById('app')` e perceba como o segundo se explica sozinho.

Resumo do capítulo

- Variáveis são caixas etiquetadas que guardam valores
- `const` para valores que não serão reatribuídos (nossa escolha padrão), `let` para os que mudam, `var` não usamos
- Os tipos principais são `string`, `number`, `boolean`, `undefined` e `null`
- Template strings com crase permitem quebras de linha e interpolação com `${}`, e são a base da renderização das nossas páginas
- Nomes descritivos em `camelCase` tornam o código legível

Para praticar

1. Crie variáveis para descrever um livro: título, autor, ano, preço e disponível (`boolean`). Escolha entre `const` e `let` justificando cada escolha.
2. Use uma template string para montar uma frase de apresentação do livro interpolando as variáveis.

3. Imprima o `typeof` de cada variável no console e confira se bate com o esperado.
4. Tente reatribuir uma `const` e leia com calma a mensagem de erro que o console mostra.

Referências

- MDN Web Docs, Declarações e variáveis: https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/First_steps/Variables
- MDN Web Docs, Template strings: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Template_literals
- W3Schools, JavaScript Variables: https://www.w3schools.com/js/js_variables.asp
- W3Schools, JavaScript Data Types: https://www.w3schools.com/js/js_datatypes.asp

Capítulo 3: Tomando decisões com if, else e switch

Programas que decidem

Até agora nossas variáveis apenas guardam valores. Mas um programa de verdade precisa tomar decisões: se a rota existe, mostra a página; senão, mostra o erro 404. Se o formulário está preenchido, envia; senão, avisa o usuário. Essa capacidade de decidir é o que chamamos de estrutura condicional.

Operadores de comparação

Antes de decidir, precisamos comparar. Os operadores de comparação sempre devolvem um boolean, ou seja, `true` ou `false`:

```
5 > 3      // true
5 < 3      // false
5 >= 5     // true
5 <= 4     // false
5 == "5"   // true (compara só o valor, converte o tipo)
5 === "5"  // false (compara valor E tipo)
5 != "5"   // false
5 !== "5"  // true
```

Regra da sala de aula: use sempre `===` e `!==`, os operadores estritos. O `==` faz conversões automáticas de tipo que geram surpresas desagradáveis. Por exemplo, `0 == ""` é `true`, o que raramente é o que você quer.

Operadores lógicos

Para combinar condições usamos três operadores:

```
// && (E): as duas condições precisam ser verdadeiras
idade >= 18 && possuiIngresso === true

// || (OU): basta uma condição ser verdadeira
diaSemana === "sabado" || diaSemana === "domingo"

// ! (NÃO): inverte o valor
!menuAberto // se menuAberto é false, vira true
```

O if e o else

A estrutura básica é: se a condição for verdadeira, executa o primeiro bloco; senão, executa o bloco do `else`:

```
const hora = 14;

if (hora < 12) {
  console.log("Bom dia");
} else if (hora < 18) {
  console.log("Boa tarde");
} else {
  console.log("Boa noite");
}
```

O `else if` permite encadear quantos testes forem necessários, e o `else` final é o plano B quando nenhum teste passou.

Onde o projeto usa condicionais sem você perceber

O truque do `||` no roteador

Olhe esta linha do nosso `main.js`:

```
let hash = window.location.hash || '#inicio';
```

O operador `||` aqui funciona como um valor padrão. Quando o usuário abre o site pela primeira vez, `window.location.hash` é uma string vazia, que o JavaScript considera um valor falso (`falsy`). Então o `||` entrega o valor da direita, `'#inicio'`. É o equivalente compacto de:

```
let hash;
if (window.location.hash) {
  hash = window.location.hash;
} else {
  hash = '#inicio';
}
```

A rota 404

O mesmo truque aparece na hora de renderizar:

```
const rota404 = { pagina: () => `<div> Página não encontrada 404 </div>` }

async function render(){
  const rotaAtual = mapaDeRotas[hash] || rota404
  await rotaAtual.pagina(app)
}
```

Se o `hash` digitado não existe no `mapaDeRotas`, a busca devolve `undefined`, que também é `falsy`, e o `||` entrega a `rota404`. Uma linha só resolve toda a lógica de página não encontrada.

Verificando se algo é uma função

Em uma das versões do roteador tínhamos este teste:

```
if (typeof mapaDeRotas[hash].acao === 'function') {
  await mapaDeRotas[hash].acao()
}
```

Traduzindo: só execute a `acao` da rota se ela existir e for de fato uma função. Sem esse `if`, rotas sem `acao` quebrariam o programa.

Valores truthy e falsy

O JavaScript considera falsos (falsy) estes valores quando aparecem em uma condição: `false`, `0`, `""` (string vazia), `null`, `undefined` e `NaN`. Todo o resto é verdadeiro (truthy), inclusive strings com conteúdo, números diferentes de zero, arrays e objetos, mesmo vazios.

É por isso que podemos escrever `if (window.location.hash)` sem comparar com nada: estamos perguntando se a string tem conteúdo.

O switch case

Quando comparamos a mesma variável contra vários valores exatos, o `switch` fica mais legível que uma escada de `else if`:

```
const rota = "#contato";

switch (rota) {
  case "#home":
    console.log("Mostrando a página inicial");
    break;
  case "#sobre":
    console.log("Mostrando a página sobre");
    break;
  case "#contato":
    console.log("Mostrando a página de contato");
    break;
  default:
    console.log("Página não encontrada 404");
}
```

Três pontos de atenção:

1. O `switch` compara com `===`, ou seja, valor e tipo
2. O `break` encerra o caso; sem ele a execução vaza para o caso seguinte
3. O `default` é o equivalente do `else`, executado quando nenhum caso combina

Curiosidade importante para o nosso projeto: nas primeiras aulas o roteamento poderia ter sido feito com um `switch` gigante testando cada hash. Nós escolhemos outro caminho, o objeto `mapaDeRotas`, porque adicionar uma rota nova vira só adicionar um item no array, sem mexer no roteador. Guarde essa comparação, ela volta no Capítulo 9.

O operador ternário

Para decisões curtas que produzem um valor, existe a forma compacta:

```
const mensagem = idade >= 18 ? "Pode entrar" : "Entrada não permitida";
```

Leia assim: condição, interrogação, valor se verdadeiro, dois pontos, valor se falso. Use apenas para casos simples; se a lógica cresce, volte para o `if` normal.

Resumo do capítulo

- Condicionais permitem que o programa escolha caminhos diferentes
- Use sempre `===` e `!==` nas comparações
- O `||` serve como valor padrão e é usado duas vezes no coração do nosso roteador
- Valores falsy: `false`, `0`, `""`, `null`, `undefined`, `NaN`
- O `switch` é uma alternativa legível para comparar uma variável contra vários valores exatos
- O ternário resolve decisões curtas em uma linha

Para praticar

1. Escreva uma função que recebe uma hora (0 a 23) e devolve a saudação correta usando `if` e `else if`.
2. Reescreva o exercício 1 usando `switch` (dica: use `switch (true)` ou faixas de valores com `Math.floor`).
3. Simule o comportamento da rota 404: crie um objeto com três rotas, busque uma rota que não existe e use `||` para devolver uma mensagem padrão.
4. Teste no console cada valor falsy dentro de um `if` e confirme que o bloco não executa.

Referências

- MDN Web Docs, `if...else`: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/if...else>
- MDN Web Docs, `switch`: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/switch>
- MDN Web Docs, Operadores lógicos: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/Logical_OR
- W3Schools, JavaScript `if else`: https://www.w3schools.com/js/js_if_else.asp
- W3Schools, JavaScript `switch`: https://www.w3schools.com/js/js_switch.asp

Capítulo 4: Repetindo tarefas com for e while

Por que repetir

Imagine montar o menu do site escrevendo um `<li>` de cada vez para Home, Cadastro, Sobre, Serviços e Contato. Agora imagine que o cliente pede uma sexta página. E uma sétima. Copiar e colar código é o caminho mais curto para o bug, porque toda alteração precisa ser repetida em vários lugares.

Os laços de repetição (loops) resolvem isso: escrevemos a instrução uma vez e mandamos o JavaScript repetir quantas vezes for preciso.

O laço for clássico

A estrutura tem três partes separadas por ponto e vírgula: inicialização, condição de continuação e incremento.

```
for (let i = 0; i < 5; i++) {  
  console.log("Volta número " + i);  
}
```

Lendo em voz alta: crie um contador `i` começando em zero; enquanto `i` for menor que 5, execute o bloco; ao final de cada volta, some 1 ao contador. O resultado imprime as voltas 0, 1, 2, 3 e 4.

Detalhe que sempre gera pergunta em aula: por que começar do zero? Porque as posições de um array começam do zero, como vimos ao estudar arrays. O primeiro item é `lista[0]`. Então o par `let i = 0` com `i < lista.length` percorre o array inteiro sem estourar o limite.

O for no projeto: a página de Serviços

A página de Serviços do App Livros é o exemplo perfeito. Temos um array de objetos com os detalhes de cada card:

```
const detalhes = [  
  {  
    titulo: 'Jogo das quartas de final da copa do mundo de 2002',  
    descricao: 'xxxxxxxxxxxxxxxxxxxx',  
    imagem: 'src/img/2002_1.webp'  
  },  
  {  
    titulo: 'Jogo especial',  
    descricao: 'xxxxxxxxxxxxxxxxxxxx',  
    imagem: 'src/img/2002_2.jpg'  
  },  
  // ... outros itens  
]
```

E um `for` que transforma cada objeto em um card de HTML:

```
function servicos(app){
  cardServico += `<div class="bem-grid-auto">`
  for(let i = 0; i < detalhes.length; i++){
    cardServico += `
      <div class="bem-card">
        
        <div class="bem-card__body">
          <h3 class="bem-card__title">${detalhes[i].titulo}</h3>
          <p>${detalhes[i].descricao}</p>
        </div>
      </div>
    `
  }
  cardServico += `</div>`
  app.innerHTML = cardServico
}
```

Repare no padrão: uma variável acumuladora (`cardServico`) começa com a abertura do grid, o laço concatena um card por volta usando `detalhes[i]` para acessar o item da vez, e no final fechamos o grid e jogamos tudo no `innerHTML` . Quatro cards ou quarenta, o código é o mesmo. Para adicionar um serviço novo, basta adicionar um objeto no array.

O for...of

Quando não precisamos do índice numérico, o `for...of` percorre diretamente os valores e deixa o código mais limpo. É ele que usamos no `main.js` para montar o mapa de rotas:

```
const mapaDeRotas = {}
for (const rota of roteador) {
  mapaDeRotas[rota.url] = rota
}
```

Leia assim: para cada `rota` dentro do array `roteador` , crie no objeto `mapaDeRotas` uma chave com a url dessa rota apontando para a rota inteira. Em três linhas transformamos um array em um objeto de consulta rápida. Esse trecho é o coração do roteador e será destrinchado no Capítulo 9.

O laço while

O `while` repete enquanto a condição for verdadeira, sem prometer quantas voltas serão. Use quando o número de repetições não é conhecido de antemão:

```
let tentativas = 0;
let senhaCorreta = false;

while (!senhaCorreta && tentativas < 3) {
  tentativas++;
  // aqui pediríamos a senha ao usuário
  console.log("Tentativa número " + tentativas);
  // senhaCorreta receberia o resultado da verificação
}
```

Cuidado clássico: se a condição nunca ficar falsa, o laço roda para sempre e trava a página. Todo `while` precisa de algo dentro do bloco que caminhe para o fim, como o `tentativas++` acima.

Existe também o `do...while`, que executa o bloco pelo menos uma vez antes de testar a condição:

```
let resposta;
do {
  resposta = "s"; // simulando uma entrada do usuário
} while (resposta !== "s" && resposta !== "n");
```

break e continue

Duas palavras controlam o laço por dentro:

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // pula esta volta e vai para a próxima
  if (i === 7) break;    // encerra o laço de vez
  console.log(i);        // imprime 0 1 2 4 5 6
}
```

for tradicional ou métodos de array?

No Capítulo 5 vamos conhecer o método `.map()`, que a navbar usa para gerar os itens do menu. Nos comentários do arquivo `navbar.js` deixamos registrada a versão com `for` para comparação:

```
/*
for(let i = 0; i < item_menu.length; i++){
  `<li class="bem-navbar__item">
    <a href="${item_menu[i].url}" class="bem-navbar__link">${item_menu[i].label}</a>
  </li>`
}
*/
```

Os dois resolvem o mesmo problema. O `for` dá controle total (índice, break, continue), enquanto o `.map()` é mais declarativo: diz o que fazer com cada item e devolve um array novo pronto. Saber os dois é obrigatório; escolher qual usar é questão de contexto e legibilidade.

Resumo do capítulo

- Laços eliminam código repetido: escreva uma vez, execute muitas

- O `for` clássico tem inicialização, condição e incremento, e casa perfeitamente com arrays indexados do zero
- O padrão acumulador (string que cresce a cada volta) é como a página de Serviços monta seus cards
- O `for...of` percorre valores diretamente e é usado para montar o `mapaDeRotas`
- O `while` repete enquanto a condição valer, e exige cuidado para não virar laço infinito
- `break` encerra o laço, `continue` pula para a próxima volta

Para praticar

1. Crie um array com cinco títulos de livros e use um `for` clássico para imprimir cada um numerado no console.
2. Refaça o exercício 1 com `for...of`.
3. Adicione um quinto objeto ao array `detalhes` da página de Serviços e confirme que o card novo aparece sem mudar o laço.
4. Escreva um `while` que soma os números de 1 a 100 e imprime o total.
5. Desafio: usando o padrão acumulador, gere uma tabela HTML (string) a partir de um array de objetos com nome e preço de livros.

Referências

- MDN Web Docs, Laços e iterações: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Loops_and_iteration
- MDN Web Docs, `for...of`: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/for...of>
- W3Schools, JavaScript For Loop: https://www.w3schools.com/js/js_loop_for.asp
- W3Schools, JavaScript While Loop: https://www.w3schools.com/js/js_loop_while.asp

Capítulo 5: Arrays e objetos, organizando os dados da aplicação

Dados que andam juntos

Uma variável guarda um valor. Mas o mundo real raramente é um valor só: um serviço tem título, descrição e imagem; uma rota tem url, label e a função da página. Precisamos de estruturas que agrupem dados relacionados. São elas o objeto e o array, e a combinação das duas sustenta o App Livros inteiro.

Objetos: dados com nome

O objeto agrupa pares de chave e valor entre chaves:

```
const rota = {  
  url: '#sobre',  
  label: 'Sobre',  
  pagina: sobre  
}
```

Este objeto existe de verdade no arquivo `sobre.js` do projeto. Ele descreve tudo que o roteador precisa saber sobre a página Sobre: o endereço (`url`), o texto do menu (`label`) e a função que desenha a página (`pagina`). Repare que um valor pode ser até uma função, e isso é fundamental para a nossa arquitetura.

Acessamos os valores de duas formas:

```
// notação de ponto, quando sabemos o nome da chave  
console.log(rota.url);      // "#sobre"  
console.log(rota.label);    // "Sobre"  
  
// notação de colchetes, quando o nome da chave está em uma variável  
const chave = "url";  
console.log(rota[chave]);    // "#sobre"
```

A notação de colchetes parece exótica até você ver o roteador usando ela:

```
mapaDeRotas[rota.url] = rota;  // cria a chave dinamicamente  
const rotaAtual = mapaDeRotas[hash]; // busca pela variável hash
```

Sem a notação de colchetes não haveria roteamento dinâmico.

Arrays: dados em fila

O array guarda uma lista ordenada de valores entre colchetes, acessados pela posição, que começa em zero:

```
const frutas = ["maçã", "banana", "uva"];
console.log(frutas[0]);      // "maçã"
console.log(frutas.length);  // 3
```

A combinação poderosa: array de objetos

O padrão que mais aparece em aplicações reais é a lista de coisas estruturadas, ou seja, um array de objetos. É assim que a página de Serviços descreve seus cards:

```
const detalhes = [
  {
    titulo: 'Jogo das quartas de final da copa do mundo de 2002',
    descricao: 'xxxxxxxxxxxxxxxxxxxx',
    imagem: 'src/img/2002_1.webp'
  },
  {
    titulo: 'Camisa azul',
    descricao: 'xxxxxxxxxxxxxxxxxxxx',
    imagem: 'src/img/2002_3.jpg'
  }
]
```

E é exatamente assim que o arquivo `rotas.js` descreve a aplicação inteira: um array em que cada item é o objeto de uma página:

```
import home from '../paginas/home.js'
import servicos from '../paginas/servicos.js'
import sobre from '../paginas/sobre.js'
import contato from '../paginas/contato.js'
import telaCadastro from '../paginas/formCad.js'

const roteador = [
  home,
  telaCadastro,
  sobre,
  servicos,
  contato,
]

export default roteador;
```

Para acessar dados aninhados, encadeamos as notações:

```
console.log(detalhes[1].titulo);  // "Camisa azul"
console.log(roteador[0].url);     // "#home"
```

Leia de dentro para fora: pegue o item da posição 1 do array, depois a chave `titulo` desse objeto.

Métodos de array: o .map() da navbar

Arrays vêm com métodos prontos que recebem uma função e aplicam a cada item. O mais importante do nosso projeto é o `.map()`, que transforma cada item em outra coisa e devolve um array novo do mesmo tamanho.

A navbar usa o `.map()` para transformar cada rota em um item de menu:

```
function navbar(item_menu){
  const navbar = document.getElementById('navbar');
  navbar.innerHTML = `<nav class="bem-navbar">
    <a href="#" class="bem-navbar__brand">Brand</a>
    <ul class="bem-navbar__menu">
      ${
        item_menu.map((item)=>{
          return `<li class="bem-navbar__item">
            <a href="${item.url}" class="bem-navbar__link">${item.label}</a>
          </li>`
        })
      }
    </ul>
  </nav>`.replaceAll(',','');
}
```

O raciocínio: o array `roteador` entra, o `.map()` transforma cada objeto de rota na string de um `<li>`, e a template string interpola o resultado dentro do `<ul>`. Adicionou uma rota nova no `rotas.js`? O menu ganha o item automaticamente, sem tocar na navbar. Isso é programar orientado a dados.

Detalhe de aula que rendeu boa discussão: quando um array é interpolado numa template string, o JavaScript junta os itens separando por vírgula. Por isso o `.replaceAll(',','')` no final, que remove as vírgulas indesejadas entre os `<li>`. Uma alternativa mais elegante é usar `.join('')` no resultado do map, que junta sem separador:

```
item_menu.map((item) => `<li>...</li>`).join('')
```

Outros métodos que valem conhecer:

```
const numeros = [1, 2, 3, 4, 5];

// filter: devolve só os itens que passam no teste
const pares = numeros.filter((n) => n % 2 === 0); // [2, 4]

// find: devolve o primeiro item que passa no teste
const maiorQueTres = numeros.find((n) => n > 3); // 4

// forEach: executa algo para cada item, sem devolver nada
numeros.forEach((n) => console.log(n));

// push: adiciona no final
numeros.push(6);
```

O objeto como mapa de consulta

Fechamos o capítulo com a jogada mais inteligente do roteador. Buscar uma rota dentro de um array exigiria percorrer item por item. Em vez disso, convertamos o array em um objeto onde a chave é a própria url:

```
const mapaDeRotas = {}
for (const rota of roteador) {
  mapaDeRotas[rota.url] = rota
}
```

O resultado é um objeto assim:

```
{
  "#home": { url: "#home", label: "Home", pagina: home },
  "#sobre": { url: "#sobre", label: "Sobre", pagina: sobre },
  // ...
}
```

Agora encontrar a rota do hash atual é acesso direto, `mapaDeRotas[hash]`, sem laço nenhum. Essa técnica de indexar dados por uma chave é usada em sistemas do mundo inteiro.

Resumo do capítulo

- Objetos agrupam dados com nome (chave e valor); arrays guardam listas ordenadas
- A notação de colchetes permite chaves dinâmicas, e é ela que viabiliza o roteador
- O array de objetos é o padrão central do projeto: `detalhes` nos serviços e `roteador` nas rotas
- O `.map()` transforma dados em HTML e faz o menu se montar sozinho a partir das rotas
- Converter um array em objeto de consulta (`mapaDeRotas`) troca busca por acesso direto

Para praticar

1. Crie um array de objetos com quatro livros (título, autor, preço) e imprima o título do terceiro livro.
2. Use `.map()` para gerar um array de strings no formato "Título custa R\$ preço".
3. Use `.filter()` para obter só os livros com preço abaixo de 50.
4. Transforme o array de livros em um objeto indexado pelo título, seguindo o padrão do `mapaDeRotas`.
5. Adicione uma nova rota fictícia ao array `roteador` do projeto e observe o menu ganhar o item sozinho.

Referências

- MDN Web Docs, Trabalhando com objetos: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Working_with_objects

- MDN Web Docs, Array: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array
- MDN Web Docs, Array.prototype.map(): https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array/map
- W3Schools, JavaScript Objects: https://www.w3schools.com/js/js_objects.asp
- W3Schools, JavaScript Arrays: https://www.w3schools.com/js/js_arrays.asp

Capítulo 6: Funções, com retorno e sem retorno

Empacotando comportamento

Se variáveis guardam valores, funções guardam comportamento. Uma função é um bloco de código com nome que pode ser executado quantas vezes quisermos, de onde quisermos. No App Livros, cada página é uma função, o roteador é uma função, a captura do formulário é uma função. Dominar funções é dominar o projeto.

Declarando e chamando

```
// declaração
function saudacao() {
  console.log("Bem-vindo ao App Livros");
}

// chamada (execução)
saudacao();
saudacao(); // pode chamar quantas vezes quiser
```

Atenção para a diferença entre a função e a chamada dela. `saudacao` (sem parênteses) é a função em si, um valor que pode ser guardado e passado adiante. `saudacao()` (com parênteses) executa a função agora. Essa diferença é vital no projeto: quando escrevemos `pagina: sobre` no objeto de rota, passamos a função sem executar, para o roteador executar depois, na hora certa.

Parâmetros: dando entrada para a função

Parâmetros tornam a função reutilizável para dados diferentes:

```
function saudacao(nome) {
  console.log(`Bem-vindo, ${nome}`);
}

saudacao("Ana");    // Bem-vindo, Ana
saudacao("Bruno");  // Bem-vindo, Bruno
```

No projeto, as funções de página recebem o elemento onde devem se montar:

```
function sobre(app){
  const sobre = `<h1> Esta é página Sobre </h1>
  <p>Este site é um exemplo de SPA usando JavaScript puro</p>
  `;

  app.innerHTML = sobre
}
```

Quem fornece o `app` é o roteador, na linha `rotaAtual.pagina(app)`. A página não precisa saber onde vai ser montada; ela recebe o destino de fora. Esse desacoplamento é o que torna cada página um componente independente.

Funções com retorno

Uma função pode devolver um valor para quem a chamou, usando `return`:

```
function somar(a, b) {  
  return a + b;  
}  
  
const total = somar(2, 3); // total vale 5
```

O `return` também encerra a função na hora: nada depois dele executa.

No projeto, a função `cadastroCliente` do formulário de CEP é uma função com retorno. Ela busca os dados na API e devolve o resultado para quem chamou:

```
async function cadastroCliente(cep){  
  try {  
    const response = await fetch(`https://viacep.com.br/ws/${cep}/json/`);  
    const result = await response.json();  
    return result  
  } catch (error) {  
    console.error(error);  
  }  
}  
  
// quem chama recebe o retorno:  
const dados = await cadastroCliente(event.target.value)
```

Funções sem retorno

Nem toda função devolve algo. Muitas existem pelo efeito que causam: desenhar na tela, registrar um evento, imprimir no console. Chamamos isso de efeito colateral. Quando não há `return`, a função devolve `undefined` automaticamente.

A função `sobre` acima é um exemplo: ela não devolve nada, seu trabalho é alterar o `innerHTML` do elemento. A `navbar` também: recebe as rotas e desenha o menu. Compare:

- Com retorno: `cadastroCliente(cep)` calcula e entrega um valor, quem chamou decide o que fazer
- Sem retorno: `sobre(app)` age diretamente sobre a página, não entrega nada

Saber classificar suas funções nessas duas famílias ajuda muito na hora de montar o quebra-cabeça de um sistema.

Arrow functions

Desde 2015 existe uma sintaxe curta, a arrow function, muito usada em callbacks:


```
// função tradicional
function dobrar(n) {
  return n * 2;
}

// arrow equivalente
const dobrar = (n) => {
  return n * 2;
}

// arrow compacta: sem chaves, o retorno é implícito
const dobrar = (n) => n * 2;
```

No projeto elas aparecem em vários pontos:

```
// no map da navbar
item_menu.map((item)=>{
  return `<li class="bem-navbar__item">
    <a href="${item.url}" class="bem-navbar__link">${item.label}</a>
  </li>`
})

// na rota 404, com retorno implícito
const rota404 = { pagina: () => `<div> Página não encontrada 404 </div>` }

// no listener de mudança de hash
window.addEventListener("hashchange", ()=>{
  hash = window.location.hash;
  render();
})
```

Funções como valores: o coração da arquitetura

Aqui está o conceito mais importante do capítulo. Em JavaScript, função é um valor como outro qualquer: pode ser guardada em variável, colocada dentro de objeto e passada como argumento para outra função (quando passada assim, chamamos de callback).

Olhe de novo o objeto que cada página exporta:

```
export default {
  url: '#sobre',
  label: 'Sobre',
  pagina: sobre
}
```

A chave `pagina` guarda a função `sobre` sem executar. O roteador guarda todos esses objetos no `mapaDeRotas` e, só quando o usuário navega, executa a função da rota atual:

```
async function render(){
  const rotaAtual = mapaDeRotas[hash] || rota404
  await rotaAtual.pagina(app)
}
```

Pare e aprecie: o roteador não conhece nenhuma página especificamente. Ele só sabe que toda rota tem uma função `pagina` que recebe o `app`. Esse contrato entre as partes é o que permite adicionar páginas novas sem tocar no roteador. Em engenharia de software isso se chama inversão de controle, e você acabou de implementar uma.

Escopo: onde cada variável enxerga

Variáveis declaradas dentro de uma função só existem dentro dela:

```
function contato(app) {  
  const paginadecontato = `...`; // só existe aqui dentro  
  app.innerHTML = paginadecontato;  
}  
console.log(paginadecontato); // erro: não existe aqui fora
```

Já variáveis declaradas fora, no escopo do módulo, são visíveis pelas funções de dentro, como o array `detalhes` que a função `servicos` usa. A regra prática: declare cada variável no menor escopo possível.

Resumo do capítulo

- Funções empacotam comportamento reutilizável; parâmetros são suas entradas, `return` é sua saída
- Função sem parênteses é valor, com parênteses é execução; o roteador depende dessa diferença
- Funções com retorno entregam valores (`cadastroCliente`), funções sem retorno causam efeitos (`sobre` , `navbar`)
- Arrow functions são a forma curta, dominante em callbacks
- Guardar funções em objetos (`pagina: sobre`) e executá-las depois é a base da arquitetura de rotas do projeto
- Cada variável deve viver no menor escopo possível

Para praticar

1. Escreva uma função com retorno que recebe um preço e devolve o valor com 10 por cento de desconto.
2. Escreva uma função sem retorno que recebe um texto e o insere em um elemento da página via `innerHTML`.
3. Converta as duas para arrow functions.
4. Crie um objeto `botao` com as chaves `label` e `aoClicar`, onde `aoClicar` é uma função. Depois execute `botao.aoClicar()`.
5. Crie uma nova página para o projeto seguindo o padrão: função que recebe `app` e objeto exportado com `url`, `label` e `pagina`.

Referências

- MDN Web Docs, Funções: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Functions>
- MDN Web Docs, Arrow functions: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Functions/Arrow_functions
- MDN Web Docs, Callbacks: https://developer.mozilla.org/pt-BR/docs/Glossary/Callback_function
- W3Schools, JavaScript Functions: https://www.w3schools.com/js/js_functions.asp
- W3Schools, JavaScript Function Parameters: https://www.w3schools.com/js/js_function_parameters.asp

Capítulo 7: DOM e eventos de escuta

O JavaScript conversa com a página

Até aqui aprendemos a linguagem. Agora vamos ao motivo de ela existir no navegador: manipular a página e reagir ao usuário. A ponte entre o JavaScript e o HTML se chama DOM, Document Object Model. O navegador lê o HTML e monta na memória uma árvore de objetos representando cada elemento. O JavaScript enxerga e altera essa árvore através do objeto global `document`.

Encontrando elementos

O método que usamos o curso inteiro é o `getElementById`:

```
const app = document.getElementById('app');
const formulario = document.getElementById('formulario-de-contato');
const campocep = document.getElementById("cep");
```

Ele devolve o objeto que representa o elemento com aquele id. Existem outros seletores que valem conhecer:

```
document.querySelector('.bem-card'); // primeiro elemento que casa com o seletor CSS
document.querySelectorAll('.bem-card'); // todos os que casam
```

Alterando a página: `innerHTML` e `textContent`

Encontrado o elemento, podemos trocar seu conteúdo. O `innerHTML` interpreta a string como HTML, e é assim que todas as nossas páginas se desenharam:

```
function sobre(app){
  const sobre = `<h1> Esta é página Sobre </h1>
  <p>Este site é um exemplo de SPA usando JavaScript puro</p>
  `
  app.innerHTML = sobre
}
```

Já o `textContent` insere texto puro, sem interpretar tags. Usamos ele na lista de contatos, e a escolha não é por acaso: como o texto vem do usuário, inserir como texto puro impede que alguém digite HTML ou script malicioso no formulário e ele seja executado na página. Isso previne um ataque conhecido como XSS. Regra prática: conteúdo que você controla pode ir de `innerHTML`; conteúdo digitado pelo usuário vai de `textContent`.

Criando elementos

Além de escrever HTML em string, dá para criar elementos um a um:

```
const lista = document.getElementById('lista_de_contatos');
const li = document.createElement('li');
li.textContent = "novo contato";
lista.appendChild(li);
```

É exatamente o que a página de Contato faz a cada envio do formulário: cria um `<li>`, preenche e anexa na lista.

Eventos: a página avisa, o código escuta

O navegador dispara eventos o tempo todo: clique, tecla pressionada, formulário enviado, campo que perdeu o foco, hash da URL que mudou. Nós registramos funções para serem chamadas quando o evento acontecer. Essas funções são os ouvintes (listeners), e o registro é feito com `addEventListener`:

```
elemento.addEventListener("nomeDoEvento", funcaoQueReage);
```

Repare: passamos a função sem parênteses. Não estamos executando agora; estamos entregando a função para o navegador executar quando o evento ocorrer. É o conceito de callback do capítulo anterior em ação.

Os três eventos do projeto

submit: o formulário de contato

```
async function capturarFormulario(){
  const formulario = document.getElementById('formulario-de-contato');
  formulario.addEventListener("submit", function(event){
    event.preventDefault();
    const lista = document.getElementById('lista_de_contatos');
    const li = document.createElement('li');
    const assunto = event.target[0].value;
    const email = event.target[1].value;
    const mensagem = event.target[2].value;
    li.textContent = `O Assunto é ${assunto}
    e o email é ${email}
    e a mensagem é ${mensagem}`;
    lista.appendChild(li);
    event.target[0].value = '';
    event.target[1].value = '';
    event.target[2].value = '';
  })
}
```

Três pontos para destrinchar:

1. O parâmetro `event` é um objeto que o navegador entrega para o ouvinte com tudo sobre o evento ocorrido.

2. `event.preventDefault()` cancela o comportamento padrão. O padrão do submit é recarregar a página, e recarregar mataria a nossa SPA. Essa linha é obrigatória em formulários de SPA.
3. `event.target` é o elemento que disparou o evento, no caso o formulário. Os índices `[0]`, `[1]`, `[2]` acessam os campos na ordem. Também poderíamos usar `document.getElementById('assunto').value`, como está registrado nos comentários do arquivo.

blur: o CEP que se completa sozinho

O evento `blur` dispara quando um campo perde o foco, ou seja, quando o usuário clica fora dele depois de digitar. Na tela de cadastro, é o gatilho perfeito para buscar o endereço assim que a pessoa termina de digitar o CEP:

```
async function capturacep(){
  const campocep = document.getElementById("cep")
  campocep.addEventListener("blur", async (event)=>{
    const dados = await cadastroCliente(event.target.value)
    document.getElementById("logradouro").value = dados.logradouro
    document.getElementById("bairro").value = dados.bairro
    document.getElementById("localidade").value = dados.localidade
    document.getElementById("estado").value = dados.estado
  })
}
```

Aqui `event.target.value` é o texto digitado no campo de CEP. Os detalhes do `async` e do `await` ficam para os Capítulos 10 e 11; por ora, registre o padrão: evento dispara, ouvinte lê o valor, busca os dados e preenche os outros campos.

hashchange: a navegação da SPA

O terceiro evento não vem de um elemento, mas da própria janela. O `hashchange` dispara toda vez que a parte após o `#` da URL muda:

```
window.addEventListener("hashchange", ()=>{
  hash = window.location.hash;
  render();
})
```

Este ouvinte é literalmente o motor da navegação do App Livros: usuário clica no menu, o hash muda, o evento dispara, o roteador redesenha a página. O Capítulo 9 é inteiro sobre ele.

Uma armadilha clássica da SPA

Atenção a um detalhe de ordem que pegou a turma em aula: só dá para registrar um ouvinte em um elemento que já existe no DOM. Como nossas páginas são desenhadas dinamicamente, o `addEventListener` do formulário precisa rodar depois do `innerHTML` que cria o formulário:

```
async function contato(app) {  
  const paginadecontato = `... o formulário ...`  
  app.innerHTML = paginadecontato; // primeiro desenha  
  await capturarFormulario()       // depois escuta  
}
```

Se invertermos a ordem, `getElementById` devolve `null` e o código quebra. Guarde esse padrão: desenhar primeiro, escutar depois. Ele é o motivo de cada página chamar sua própria função de eventos no final.

Resumo do capítulo

- O DOM é a representação da página que o JavaScript manipula via `document`
- `getElementById` encontra elementos; `innerHTML` desenha HTML; `textContent` insere texto seguro
- `addEventListener(evento, callback)` registra funções que reagem ao usuário
- `event.preventDefault()` impede o recarregamento no submit e mantém a SPA viva
- `event.target` dá acesso ao elemento que disparou o evento e seus valores
- O projeto usa `submit` (contato), `blur` (CEP) e `hashchange` (navegação)
- Ordem obrigatória em páginas dinâmicas: desenhar primeiro, escutar depois

Para praticar

1. Adicione um botão na página Home e um ouvinte de `click` que imprime uma mensagem no console.
2. No formulário de contato, adicione uma validação: se o assunto estiver vazio, não adicione o item na lista.
3. Adicione um ouvinte de `input` no campo de mensagem que mostra em tempo real quantos caracteres foram digitados.
4. Experimente remover o `event.preventDefault()` do formulário e observe o que acontece com a página. Depois devolva a linha.
5. Explique com suas palavras por que o `capturarFormulario` precisa ser chamado depois do `innerHTML`.

Referências

- MDN Web Docs, Introdução ao DOM: https://developer.mozilla.org/pt-BR/docs/Web/API/Document_Object_Model/Introduction
- MDN Web Docs, `addEventListener`: <https://developer.mozilla.org/pt-BR/docs/Web/API/EventTarget/addEventListener>
- MDN Web Docs, `Event.preventDefault`: <https://developer.mozilla.org/pt-BR/docs/Web/API/Event/preventDefault>
- W3Schools, JavaScript HTML DOM: https://www.w3schools.com/js/js_htmlDOM.asp

- W3Schools, JavaScript Events: https://www.w3schools.com/js/js_events.asp

Capítulo 8: ES Modules e componentização

Do arquivo único ao projeto organizado

Nas primeiras aulas todo o código cabia em um arquivo. Mas o projeto cresceu: navbar, cinco páginas, rotas, roteador. Manter tudo junto vira um arquivo gigante onde ninguém acha nada e todo mundo esbarra em todo mundo. A solução do JavaScript moderno são os ES Modules: cada arquivo é um módulo que declara o que oferece (export) e o que precisa (import).

Ativando os módulos

Basta um atributo no script do `index.html`:

```
<script src="src/js/main.js" type="module"></script>
```

Com `type="module"`, o navegador entende os comandos `import` e `export` e carrega os arquivos em cadeia a partir do `main.js`. É por causa desse atributo que o projeto exige um servidor HTTP, como vimos no Capítulo 1.

export default e import

Cada arquivo de página do projeto termina exportando seu objeto de rota:

```
// sobre.js
function sobre(app){
  const sobre = `<h1> Esta é página Sobre </h1>
  <p>Este site é um exemplo de SPA usando JavaScript puro</p>
  `

  app.innerHTML = sobre
}

export default {
  url: '#sobre',
  label: 'Sobre',
  pagina: sobre
}
```

O `export default` diz: quando alguém importar este arquivo, entregue este valor. E quem importa escolhe o nome que quiser para recebê-lo:

```
// rotas.js
import home from '../paginas/home.js'
import servicos from '../paginas/servicos.js'
import sobre from '../paginas/sobre.js'
import contato from '../paginas/contato.js'
import telaCadastro from '../paginas/formCad.js'

const roteador = [
  home,
  telaCadastro,
  sobre,
  servicos,
  contato,
]

export default roteador;
```

Repare no detalhe dos caminhos: `../paginas/home.js` significa "suba uma pasta e entre em paginas". Caminhos relativos com `./` e `../` são obrigatórios nos imports do navegador, e a extensão `.js` também.

Além do default, existe o export nomeado, útil quando um arquivo oferece várias coisas:

```
// exportando
export function formatar() { ... }
export const VERSAO = "1.0";

// importando com chaves e o nome exato
import { formatar, VERSAO } from './utilitarios.js';
```

No projeto usamos `export default` porque cada arquivo de componente entrega uma coisa só.

O que é um componente

Componente é um pedaço autocontido da interface: ele sabe se desenhar e sabe reagir aos seus próprios eventos. No App Livros, cada página é um componente que segue o mesmo contrato:

1. Uma função que recebe o elemento `app` e se desenha nele via `innerHTML`
2. Se a página tem interatividade, ela mesma registra seus ouvintes depois de se desenhar
3. Um `export default` de objeto com `url`, `label` e `pagina`

Veja o contrato completo na tela de cadastro:

```

async function telaCadastro(app){
  const formulario = `
    <form id="cadastroCliente" >
      <label for="cep">CEP</label>
      <input type="text" id="cep">
      <label for="logradouro">logradouro</label>
      <input type="text" id="logradouro">
      <label for="bairro">bairro</label>
      <input type="text" id="bairro">
      <label for="localidade">localidade</label>
      <input type="text" id="localidade">
      <label for="estado">estado</label>
      <input type="text" id="estado">
    </form>
  `

  app.innerHTML = formulario;
  await capturacep()
}

export default {
  url: '#cep',
  label: 'Cadastro',
  pagina: telaCadastro
}

```

A evolução para o componente auto montável

Vale registrar a história dessa arquitetura, porque ela evoluiu entre as aulas e o histórico do Git guarda os dois momentos.

Na primeira versão, as funções de página apenas devolviam a string de HTML, e o roteador fazia o `innerHTML` e depois executava uma `acao` separada para registrar os eventos:

```

// versão antiga (registrada nos comentários do main.js)
app.innerHTML = rotaAtual.pagina()
if (typeof mapaDeRotas[hash].acao === 'function') {
  await mapaDeRotas[hash].acao()
}

```

Funcionava, mas o conhecimento sobre a página ficava espalhado: o roteador precisava saber que existia uma `acao` e lembrar de chamá-la. Na refatoração (commit "componente auto montavel"), invertamos: a função da página passou a receber o `app` e cuidar de tudo sozinha, desenho e eventos. O roteador encolheu para uma linha:

```

// versão atual
await rotaAtual.pagina(app)

```

Essa é uma lição de arquitetura que vai muito além deste projeto: cada componente deve ser dono do seu próprio ciclo de vida. Quem usa o componente não precisa conhecer seus detalhes internos. É o mesmo princípio dos componentes do React e do Vue.

A navbar orientada a dados

A navbar é o componente que amarra tudo. Ela não conhece nenhuma página; recebe o array de rotas e desenha um link para cada uma:

```
// main.js
import navbar from "../components/navbar/navbar.js";
import roteador from "../components/rotas/rotas.js";
navbar(roteador);
```

O fluxo completo de dependências do projeto fica assim:

```
index.html
  carrega main.js
    importa rotas.js (que importa todas as paginas)
    importa navbar.js (que desenha o menu a partir das rotas)
    monta o mapaDeRotas e escuta o hashchange
```

Quer criar uma página nova? O passo a passo é sempre o mesmo, e não toca em nenhum arquivo existente além do `rotas.js`:

1. Crie `src/js/components/paginas/minhapagina.js` seguindo o contrato
2. Importe e adicione no array do `rotas.js`
3. Pronto: o menu ganha o link e a rota funciona

Organização de pastas e nomenclatura

A regra do projeto: uma pasta por componente, dentro de `components`. O CSS segue a metodologia BEM (Block, Element, Modifier), visível nas classes como `bem-navbar__item` e `bem-card__title`: o bloco é o componente, o elemento vem após dois underlines, e modificadores após dois hífens (`bem-btn--primary`). Essa disciplina de nomes evita que o estilo de um componente vaze para outro.

Resumo do capítulo

- ES Modules dividem o projeto em arquivos com `import` e `export`, ativados pelo `type="module"`
- `export default` entrega o valor principal do arquivo; caminhos relativos e extensão `.js` são obrigatórios
- Componente é um pedaço autocontido: se desenha e escuta seus próprios eventos
- O contrato das páginas: função que recebe `app` mais objeto com `url`, `label` e `pagina`
- A refatoração para componentes auto montáveis simplificou o roteador para uma linha
- Adicionar página nova não exige tocar no roteador nem na navbar

Para praticar

1. Crie um componente `footer` que desenha um rodapé e importe no `main.js`.
2. Crie uma página nova "Autores" seguindo o contrato completo e registre no `rotas.js`.

3. Desenhe no papel o diagrama de imports do projeto, seta por seta, começando do `index.html`.
4. Compare no Git as duas versões do roteador (antes e depois do commit "componente auto montavel") e escreva um parágrafo sobre o que melhorou.

Referências

- MDN Web Docs, Módulos JavaScript: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Modules>
- MDN Web Docs, import: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/import>
- MDN Web Docs, export: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/export>
- W3Schools, JavaScript Modules: https://www.w3schools.com/js/js_modules.asp
- Metodologia BEM: <https://getbem.com/>

Capítulo 9: SPA e roteamento por hash

O capítulo que junta tudo

Este é o capítulo em que todas as peças anteriores se encaixam: objetos e arrays (as rotas), funções como valores (as páginas), eventos (o hashchange), laços (o mapa de rotas) e condicionais (a rota 404). Ao final, você vai entender cada linha do `main.js`, o arquivo mais importante do projeto.

O problema: navegar sem recarregar

Em um site tradicional, cada clique em um link pede uma página nova ao servidor, e o navegador recarrega tudo: HTML, CSS, scripts. Em uma SPA queremos trocar só o conteúdo, instantaneamente, mantendo o estado da aplicação. Mas então surge a pergunta: se a página nunca muda, como o usuário navega? Como o botão voltar funciona? Como se compartilha o link de uma seção?

A resposta clássica é o hash da URL.

O que é o hash

O hash é tudo que vem depois do `#` em uma URL:

```
http://localhost:8000/#sobre
      ^^^^^ isto é o hash
```

O hash tem duas propriedades perfeitas para uma SPA:

1. Mudar o hash não recarrega a página. O navegador entende o `#` como uma âncora interna
2. Mudar o hash dispara o evento `hashchange` e entra no histórico do navegador, então os botões voltar e avançar funcionam de graça

O JavaScript lê o hash atual em `window.location.hash`. Se a URL é `#contato`, esse valor é a string `"#contato"`.

É por isso que todos os links do nosso menu apontam para hashes:

```
<a href="#sobre" class="bem-navbar__link">Sobre</a>
```

Clicou, o hash muda, evento dispara, nós redesenhamos. Nenhuma requisição ao servidor.

O main.js linha por linha

Este é o arquivo completo do roteador. Vamos dissecar:

```
import navbar from "../components/navbar/navbar.js";
import roteador from "../components/rotas/rotas.js";
navbar(roteador);

const app = document.getElementById('app');

const mapaDeRotas = {}
for (const rota of roteador) {
  mapaDeRotas[rota.url] = rota
}

let hash = window.location.hash || '#inicio';
render();

window.addEventListener("hashchange", () => {
  hash = window.location.hash;
  render();
})

const rota404 = { pagina: () => `<div> Página não encontrada 404 </div>` }

async function render() {
  const rotaAtual = mapaDeRotas[hash] || rota404
  await rotaAtual.pagina(app)
}
```

Passo 1: montar o menu

```
navbar(roteador);
```

Entregamos o array de rotas para a navbar, que desenha um link para cada uma com `.map()`, como vimos no Capítulo 5. O menu nasce dos mesmos dados que o roteador usa; uma fonte única de verdade.

Passo 2: capturar o palco

```
const app = document.getElementById('app');
```

Guardamos a referência ao `<main id="app">`, o palco onde toda página se apresenta.

Passo 3: indexar as rotas

```
const mapaDeRotas = {}
for (const rota of roteador) {
  mapaDeRotas[rota.url] = rota
}
```

O array vira um objeto indexado pela url. Nas aulas usamos vários `console.log` para enxergar essa transformação, e vale repetir o exercício:

```
console.log(mapaDeRotas)
console.log(mapaDeRotas["#home"])
console.log(mapaDeRotas["#home"].pagina)
```

O primeiro mostra o objeto completo, o segundo mostra uma rota, o terceiro mostra a função da página (sem executar). Encadear logs assim é uma técnica de estudo excelente para entender estruturas.

Passo 4: descobrir onde estamos

```
let hash = window.location.hash || '#inicio';
render();
```

Na primeira carga, lemos o hash da URL. Se estiver vazio (usuário abriu a raiz do site), o `||` entrega o padrão `'#inicio'`. Em seguida renderizamos pela primeira vez. Sem esse `render()` inicial, a página abriria em branco até o primeiro clique.

Passo 5: escutar a navegação

```
window.addEventListener("hashchange", () => {
  hash = window.location.hash;
  render();
})
```

O motor da SPA. Toda mudança de hash, seja por clique no menu, por edição manual da URL ou pelo botão voltar, atualiza a variável e redesenha.

Passo 6: renderizar com plano B

```
const rota404 = { pagina: () => `<div> Página não encontrada 404 </div>` }

async function render() {
  const rotaAtual = mapaDeRotas[hash] || rota404
  await rotaAtual.pagina(app)
}
```

A função `render` busca a rota do hash atual no mapa. Não achou? O `||` entrega a `rota404`, que segue o mesmo contrato (tem uma função `pagina`), então o resto do código nem percebe a diferença. Por fim, executamos a função da página passando o palco. Como algumas páginas são `async` (a de cadastro busca dados na API), o `render` é `async` e usa `await`, assunto do próximo capítulo.

Por que um mapa e não um switch

No Capítulo 3 prometemos voltar a esta comparação. O roteamento poderia ser um `switch`:


```
switch (hash) {
  case "#home": home(app); break;
  case "#sobre": sobre(app); break;
  case "#contato": contato(app); break;
  default: /* 404 */
}
```

Funciona, mas cada página nova exige editar o roteador. Com o `mapaDeRotas`, o roteador é genérico: dados novos (rotas) entram sem código novo. Essa é a diferença entre programar procedimentos e programar orientado a dados, e é uma das grandes lições do curso.

Hash routing e o mundo real

Frameworks modernos usam também a History API (`pushState`), que permite URLs sem `#`, como `/sobre`. Ela exige configuração no servidor para devolver o `index.html` em qualquer caminho. O hash routing que implementamos é a técnica mais simples e robusta, funciona em qualquer servidor estático, e é inclusive o modo de compatibilidade dos roteadores do React e do Vue. Entendendo o nosso, você entende os deles.

Resumo do capítulo

- O hash muda a URL sem recarregar a página e dispara o evento `hashchange`
- O roteador transforma o array de rotas em um objeto de consulta direta
- O fluxo: carga inicial lê o hash (com padrão via `||`), evento escuta as mudanças, `render` desenha
- A rota 404 segue o mesmo contrato das outras e entra pelo `||`
- Mapa de rotas vence o switch porque adicionar páginas não exige mexer no roteador
- Botões voltar e avançar funcionam automaticamente porque o hash entra no histórico

Para praticar

1. Abra o site, navegue por todas as páginas e observe a URL e o console a cada clique.
2. Digite na URL um hash que não existe, como `#banana`, e confirme a página 404.
3. Use os botões voltar e avançar do navegador e explique por que funcionam.
4. Adicione um `console.log(hash)` dentro do listener de `hashchange` e observe os valores.
5. Desafio: faça a rota 404 mostrar também um link de volta para a Home.

Referências

- MDN Web Docs, Window: hashchange event: https://developer.mozilla.org/pt-BR/docs/Web/API/Window/hashchange_event
- MDN Web Docs, Location.hash: <https://developer.mozilla.org/pt-BR/docs/Web/API/Location/hash>
- MDN Web Docs, History API: https://developer.mozilla.org/pt-BR/docs/Web/API/History_API
- W3Schools, Window Location: https://www.w3schools.com/js/js_window_location.asp

Capítulo 10: Assincronismo, o tempo no JavaScript

Um teste que muda a intuição

Começamos este assunto em aula com um experimento que deixamos registrado nos comentários do `main.js`. Antes de rodar, tente prever a ordem das mensagens:

```
console.log("A Primeira chamada")
setTimeout(() => {
  console.log("A Segunda Execução ou não?")
})
console.log("A Terceira execução ou não?")
```

A intuição diz: primeira, segunda, terceira. O que aparece no console:

```
A Primeira chamada
A Terceira execução ou não?
A Segunda Execução ou não?
```

A segunda mensagem foi por último, mesmo com o `setTimeout` sem tempo definido (zero milissegundos). Compare com a versão síncrona, também dos nossos testes:

```
console.log("B Primeira chamada")
function sincrono() {
  console.log("B Segunda Execução ou não?")
}
sincrono()
console.log("B Terceira execução ou não?")
```

Aqui a ordem é a esperada: primeira, segunda, terceira. Entender por que os dois exemplos se comportam diferente é entender o assincronismo.

O event loop em uma explicação de sala

O JavaScript executa uma coisa de cada vez, em uma única fila principal. Só que o navegador tem ajudantes: temporizadores, requisições de rede, leitura de arquivos. Quando pedimos algo a um ajudante (como o `setTimeout` ou o `fetch`), o JavaScript não fica parado esperando; ele entrega a tarefa, segue executando o código atual até o fim e deixa anotado o que fazer quando o ajudante terminar (o callback).

Quando o ajudante termina, o callback entra em uma fila de espera. O event loop é o porteiro que só deixa a fila de espera entrar quando a fila principal esvaziou. Por isso o `setTimeout` de zero milissegundos ainda executa por último: o código principal precisa terminar primeiro.

Por que a linguagem é assim? Porque o JavaScript nasceu para o navegador, e o navegador não pode congelar. Se uma busca na rede travasse tudo por três segundos, a página ficaria três segundos sem responder a cliques. Assíncrono significa: peça, continue trabalhando, reaja quando a resposta chegar.

Você já usava assincronismo sem saber

Olhe de novo o padrão dos eventos do Capítulo 7:

```
formulario.addEventListener("submit", function(event) { ... })
```

Isso é assincronismo puro: entregamos um callback que executa em algum momento futuro, quando o usuário decidir enviar. A diferença do `setTimeout` é só quem decide o momento: lá, o relógio; aqui, o usuário.

Promises: um valor que ainda vai chegar

Callbacks funcionam, mas encadear vários (busque isso, depois aquilo, depois aquilo outro) gera um aninhamento infernal apelidado de callback hell. A solução moderna é a Promise, um objeto que representa um valor futuro. Ela está em um de três estados:

- `pending`: ainda esperando
- `fulfilled`: deu certo, o valor chegou
- `rejected`: deu errado, temos um erro

Consumimos uma Promise com `.then()` para o sucesso e `.catch()` para o erro:

```
fetch("https://viacep.com.br/ws/01001000/json/")
  .then((response) => response.json())
  .then((dados) => console.log(dados))
  .catch((erro) => console.error(erro));
```

async e await: assíncrono com cara de síncrono

O `async/await` é uma sintaxe por cima das Promises que deixa o código com a leitura natural de cima para baixo. É o estilo que adotamos no projeto inteiro:

- `async` antes de uma função declara que ela trabalha com Promises (e faz ela sempre devolver uma Promise)
- `await` pausa a função (só ela, não a página) até a Promise resolver, e entrega o valor

O mesmo exemplo acima, no estilo do projeto:

```
async function buscarCep(cep) {
  try {
    const response = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
    const result = await response.json();
    return result
  } catch (error) {
    console.error(error);
  }
}
```

Você reconhece: é a estrutura exata da função `cadastroCliente` do `formCad.js`.

try e catch: o plano para quando dá errado

Rede falha. API sai do ar. Usuário digita CEP inexistente. Código assíncrono que fala com o mundo externo precisa de tratamento de erro, e com `async/await` usamos o bloco `try/catch`:

```
try {  
  // caminho feliz: tente executar isto  
} catch (error) {  
  // plano B: se qualquer linha do try falhar, caia aqui  
}
```

Sem o `try/catch`, um erro na rede quebraria a função e possivelmente deixaria a tela travada em um estado estranho. Com ele, capturamos o problema e decidimos o que fazer: registrar no console, mostrar mensagem ao usuário, tentar de novo.

O assincronismo espalhado pelo projeto

Repare que várias funções do projeto são `async`:

```
async function telaCadastro(app) { ... }  
async function render() {  
  const rotaAtual = mapaDeRotas[hash] || rota404  
  await rotaAtual.pagina(app)  
}
```

O `render` usa `await` ao chamar a página porque algumas páginas fazem trabalho assíncrono (a tela de cadastro registra a busca de CEP). Como o roteador não sabe qual página é qual, ele trata todas como potencialmente assíncronas. Mais uma vez o contrato uniforme entre componentes simplificando o sistema.

Uma observação honesta de sala de aula: páginas como a Home são `async` sem precisar, por padronização. Não há erro nisso, mas saiba diferenciar: o `async` só é necessário quando existe `await` dentro.

Resumo do capítulo

- O JavaScript executa uma coisa por vez; tarefas demoradas vão para ajudantes do navegador e voltam por callbacks
- O event loop só processa a fila de callbacks quando o código principal termina, por isso o `setTimeout(0)` roda por último
- Eventos de usuário já são assincronismo: callbacks para momentos futuros
- Promise é um valor futuro com três estados: pending, fulfilled e rejected
- `async/await` dá leitura natural ao código assíncrono e é o estilo do projeto
- `try/catch` é o plano B obrigatório quando se fala com o mundo externo

Para praticar

1. Rode os dois testes do início do capítulo e explique a ordem das mensagens com suas palavras.
2. Crie dois `setTimeout`, um com 2000 e outro com 1000 milissegundos, e preveja a ordem antes de rodar.
3. Escreva uma função `esperar(ms)` que devolve uma Promise que resolve depois de `ms` milissegundos (pesquise `new Promise` no MDN). Use com `await`.
4. Modifique a função de CEP para imprimir "buscando..." antes do `await` e "chegou!" depois, e observe o intervalo no console.
5. Force um erro passando uma URL inválida ao `fetch` e confirme que o `catch` captura.

Referências

- MDN Web Docs, JavaScript Assíncrono: <https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Asynchronous>
- MDN Web Docs, Event loop: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Event_loop
- MDN Web Docs, Promise: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Promise
- MDN Web Docs, async function: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/async_function
- W3Schools, JavaScript Async: https://www.w3schools.com/js/js_async.asp
- W3Schools, JavaScript Promises: https://www.w3schools.com/js/js_promise.asp

Capítulo 11: Consumindo APIs com fetch

Saindo da ilha

Até agora nossa aplicação vivia isolada: todos os dados estavam escritos no próprio código, como o array `detalhes` dos serviços. Aplicações reais buscam dados do mundo: previsão do tempo, catálogo de produtos, endereço a partir do CEP. A porta de entrada para esse mundo é a API.

O que é uma API

API significa Application Programming Interface, interface de programação de aplicações. No contexto da web, é um serviço que responde a requisições HTTP devolvendo dados, geralmente no formato JSON, em vez de páginas HTML. Pense na API como um garçom: você faz o pedido (requisição), ele leva à cozinha (servidor) e traz o prato (resposta em JSON).

A API que usamos no projeto é a ViaCEP, gratuita e sem cadastro, que devolve o endereço de qualquer CEP brasileiro. Experimente abrir no navegador:

```
https://viacep.com.br/ws/01001000/json/
```

A resposta é um texto no formato JSON:

```
{
  "cep": "01001-000",
  "logradouro": "Praça da Sé",
  "bairro": "Sé",
  "localidade": "São Paulo",
  "uf": "SP",
  "estado": "São Paulo"
}
```

JSON: o idioma das APIs

JSON, JavaScript Object Notation, é um formato de texto para troca de dados que nasceu da sintaxe de objetos do JavaScript. As diferenças para um objeto: as chaves vêm sempre entre aspas duplas e não pode haver funções nem comentários. Como é texto, ele viaja pela rede; do nosso lado, convertemos texto em objeto de verdade para trabalhar:

```
const objeto = JSON.parse(textoJson);      // texto vira objeto
const texto = JSON.stringify(objeto);      // objeto vira texto
```

O fetch

O `fetch` é a função do navegador para fazer requisições HTTP. Ele devolve uma Promise, por isso todo o Capítulo 10 foi pré-requisito deste. O padrão completo, direto do nosso `formCad.js`:

```
async function cadastroCliente(cep){
  try {
    const response = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
    const result = await response.json();
    return result
  } catch (error) {
    console.error(error);
  }
}
```

Repare que são dois `await`, e isso confunde no início:

1. `await fetch(...)` espera a resposta HTTP chegar. O que chega é um objeto `Response` com metadados (status, cabeçalhos) e o corpo ainda não lido
2. `await response.json()` lê o corpo e converte o texto JSON em objeto JavaScript. Também é assíncrono porque o corpo pode ser grande e chegar em pedaços

E repare na template string montando a URL: `https://viacep.com.br/ws/${cep}/json/`. O CEP digitado pelo usuário entra na URL por interpolação. Capítulo 2 trabalhando junto com o Capítulo 10.

A tela de cadastro completa

Agora temos repertório para ler o componente inteiro:

```

async function capturecep(){
  const campocep = document.getElementById("cep")
  campocep.addEventListener("blur", async (event)=>{
    const dados = await cadastroCliente(event.target.value)
    document.getElementById("logradouro").value = dados.logradouro
    document.getElementById("bairro").value = dados.bairro
    document.getElementById("localidade").value = dados.localidade
    document.getElementById("estado").value = dados.estado
  })
}

async function telaCadastro(app){
  const formulario = `
    <form id="cadastroCliente" >
      <label for="cep">CEP</label>
      <input type="text" id="cep">
      <label for="logradouro">logradouro</label>
      <input type="text" id="logradouro">
      <label for="bairro">bairro</label>
      <input type="text" id="bairro">
      <label for="localidade">localidade</label>
      <input type="text" id="localidade">
      <label for="estado">estado</label>
      <input type="text" id="estado">
    </form>
  `

  app.innerHTML = formulario;
  await capturecep()
}

export default {
  url: '#cep',
  label: 'Cadastro',
  pagina: telaCadastro
}

```

O fluxo completo, do clique ao endereço preenchido:

1. Usuário navega para `#cep`, o roteador chama `telaCadastro(app)`
2. O formulário é desenhado via `innerHTML` e `capturecep` registra o ouvinte de `blur` no campo de CEP (desenhar primeiro, escutar depois, Capítulo 7)
3. Usuário digita o CEP e clica fora; o `blur` dispara
4. O ouvinte chama `cadastroCliente` com o valor digitado e espera a resposta
5. Com o objeto em mãos, preenche cada campo do formulário pela propriedade correspondente

Cada linha desse componente usa um capítulo desta apostila. É o momento do curso em que os fundamentos viram aplicação de verdade.

Investigando pelo DevTools

Abra a aba Network (Rede) do F12, digite um CEP no formulário e observe a requisição aparecer. Clique nela e explore: a URL chamada, o status (200 significa sucesso, 404 não encontrado), os

cabeçalhos e a resposta crua. Essa aba é a ferramenta número um para depurar consumo de API.

Tratando o mundo real

Nosso código atual confia demais no sucesso. Dois problemas para amadurecer:

Primeiro: o `fetch` só rejeita a Promise em erro de rede. Se o servidor responder um erro HTTP (como 400 para CEP mal formatado), a Promise resolve normalmente. O teste correto usa `response.ok`:

```
const response = await fetch(url);
if (!response.ok) {
  throw new Error(`Erro HTTP: ${response.status}`);
}
```

Segundo: a ViaCEP responde `{ "erro": true }` para CEP com formato válido mas inexistente. Nesse caso `dados.logradouro` é `undefined` e o formulário fica preenchido com "undefined". O tratamento:

```
const dados = await cadastroCliente(event.target.value)
if (!dados || dados.erro) {
  alert("CEP não encontrado");
  return;
}
```

Melhorar esses dois pontos é um dos exercícios do capítulo.

Métodos HTTP: um vislumbre do próximo passo

Tudo que fizemos foi leitura, o método GET, que é o padrão do `fetch`. APIs completas também aceitam escrita, e o `fetch` recebe um segundo parâmetro de configuração:

```
await fetch("https://api.exemplo.com/livros", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ titulo: "Novo livro", autor: "Autora" })
});
```

GET busca, POST cria, PUT atualiza, DELETE remove. Esse vocabulário, chamado de padrão REST, é assunto para os refinamentos do fim do curso, quando vamos popular a aplicação de livros com dados de uma API de verdade.

Uma nota de evolução do código: depois desta aula, a função de busca deixou de morar dentro do componente e foi centralizada em uma camada de serviços (`services/api.js`), servindo tanto o CEP quanto a nova página de personagens da API Rick and Morty. Essa refatoração e o cache que nasceu sobre ela são o assunto do próximo capítulo.

Resumo do capítulo

- API é um serviço que responde requisições HTTP com dados, geralmente em JSON
- JSON é texto no formato de objeto; `response.json()` o converte em objeto utilizável

- O padrão do projeto: `async`, `try/catch`, `await fetch`, `await response.json()`, `return`
- São dois awaits porque resposta e corpo chegam em momentos distintos
- A aba Network do DevTools mostra cada requisição em detalhe
- Erros HTTP não caem no catch; teste `response.ok` e trate respostas de negócio como o `erro: true` da ViaCEP

Para praticar

1. Abra a URL da ViaCEP no navegador com seu próprio CEP e leia o JSON retornado.
2. Adicione o teste de `response.ok` e o tratamento de `dados.erro` na tela de cadastro.
3. Adicione um campo "complemento" ao formulário e preencha com o dado da API.
4. Explore outra API pública, como a PokeAPI (<https://pokeapi.co>), e imprima no console o nome de um Pokémon buscado por id.
5. Desafio: crie uma página nova que busca e lista dados de uma API pública usando o padrão acumulador do Capítulo 4 para montar os cards.

Referências

- MDN Web Docs, Usando Fetch: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API/Using_Fetch
- MDN Web Docs, Trabalhando com JSON: <https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Objects/JSON>
- MDN Web Docs, Métodos de requisição HTTP: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods>
- W3Schools, JavaScript Fetch API: https://www.w3schools.com/js/js_api_fetch.asp
- W3Schools, JSON Introduction: https://www.w3schools.com/js/js_json_intro.asp
- Documentação da ViaCEP: <https://viacep.com.br/>

Capítulo 12: Estratégias de cache e os armazenamentos do navegador

O problema que ninguém vê até medir

No capítulo anterior nossa aplicação aprendeu a buscar dados na internet. Agora ela tem um vício caro: busca os mesmos dados repetidas vezes. Abra a página de personagens da API Rick and Morty, navegue para a página 2, volte para a 1. A aba Network do DevTools mostra a verdade: a página 1 foi baixada de novo, inteirinha, mesmo que nada tenha mudado nos últimos dez segundos.

Isso desperdiça em três frentes ao mesmo tempo:

- Tempo do usuário: cada revisita espera a rede de novo
- Dados do usuário: em rede móvel, cada requisição repetida consome o pacote de internet
- Recursos do servidor: APIs públicas têm limites de uso, e cada chamada desnecessária ocupa a cota

A pergunta que guia este capítulo é simples de formular e valiosa de responder: por que buscar de novo algo que já temos? A resposta tem nome, cache, e é uma das técnicas mais importantes de toda a computação.

Cache no mundo real: você usa o dia inteiro

Cache é qualquer cópia local de um dado que custa caro buscar na origem. Antes de ver o código, repare como a ideia está em toda parte:

- O navegador guarda imagens, CSS e scripts de sites que você visita. Por isso a segunda visita a um site é sempre mais rápida que a primeira
- O Instagram e o X mostram o feed antigo na hora quando você abre o aplicativo sem internet, e atualizam quando a conexão volta. Aquele feed é um cache
- O Spotify guarda as músicas baixadas para tocar sem rede
- A Netflix instala servidores de cache dentro dos provedores de internet (as CDNs) para o filme sair de perto de você, e não do outro lado do planeta
- O seu processador tem memórias de cache (L1, L2, L3) porque buscar na memória RAM é lento demais para ele
- O DNS, que traduz nomes como google.com em endereços IP, é cacheado pelo sistema operacional para não perguntar de novo a cada clique

Um dado do mercado que justifica todo o esforço: estudos da Amazon e do Google associam cada 100 milissegundos a mais de espera a quedas mensuráveis de vendas e de engajamento. Velocidade é funcionalidade, e cache é a forma mais barata de comprá-la.

E uma frase clássica da área, atribuída a Phil Karlton, para manter a humildade: existem apenas duas coisas difíceis na computação, invalidação de cache e dar nome às coisas. Vamos encontrar as duas neste capítulo.

Os armazenamentos do navegador, um tour completo

Para ter cache no front-end, precisamos de um lugar para guardar as cópias. O navegador oferece cinco opções, cada uma com sua vocação. Conhecer as cinco é o que permite escolher bem.

Cookies

O mais antigo dos armazenamentos. Guardam textos minúsculos (por volta de 4 KB) e têm uma característica única: são enviados automaticamente ao servidor em toda requisição para aquele domínio. Por isso servem para identificação de sessão (o servidor reconhece que você é você), e por isso mesmo não servem para cache: inflariam cada requisição com dados que o servidor não pediu.

```
document.cookie = "tema=noite; max-age=31536000";
```

localStorage

Um armário de chave e valor com cerca de 5 a 10 MB por site. O que for guardado persiste para sempre, mesmo fechando o navegador, até alguém apagar. A API é síncrona e tem quatro métodos:

```
localStorage.setItem("tema", "noite"); // guardar
localStorage.getItem("tema");          // ler, devolve null se nao existir
localStorage.removeItem("tema");       // remover uma chave
localStorage.clear();                  // apagar tudo do site
```

Regra de ouro que pega todo mundo: o localStorage só guarda texto. Um objeto vira a string inútil "[object Object]". A ponte é o JSON, que conhecemos no capítulo anterior:

```
const livro = { id: 1, titulo: "Dom Casmurro" };
localStorage.setItem("livro", JSON.stringify(livro)); // objeto vira texto
const salvo = JSON.parse(localStorage.getItem("livro")); // texto vira objeto
```

Você pode inspecionar tudo que está guardado no DevTools, aba Application, seção Local Storage. Acostume-se a olhar ali enquanto estuda este capítulo.

sessionStorage

Mesma API do localStorage, métodos idênticos, mas o tempo de vida é outro: os dados morrem quando a aba fecha. Pense no localStorage como um armário em casa e no sessionStorage como o bolso durante um passeio. É ideal para rascunhos de formulário e estados temporários de navegação.

IndexedDB

Um banco de dados de verdade dentro do navegador: guarda objetos sem conversão para texto, aceita volumes na casa de centenas de MB, tem índices de busca e transações, e toda a API é assíncrona. É o que aplicações grandes usam para trabalhar offline com muitos dados (o Google Docs, por exemplo). O custo é uma API verbosa; para o nosso volume de dados, o localStorage resolve. Fica registrado como o próximo degrau quando o cache crescer.

Cache API

Feita sob medida para cache de rede: em vez de chave e texto, ela guarda pares de requisição e resposta HTTP completas, como um mini servidor dentro do navegador:

```
const cache = await caches.open("app-livros-v1");
await cache.add("https://rickandmortyapi.com/api/character/?page=1");
const resposta = await caches.match("https://rickandmortyapi.com/api/character/?page=1");
```

Ela é a parceira natural dos Service Workers, que veremos no fim do capítulo.

Tabela de decisão

Armazenamento	Capacidade	Vive até	Vocação
Cookies	~4 KB	expiração definida	sessão e identificação junto ao servidor
localStorage	~5 a 10 MB	ser apagado	preferências e cache simples de dados
sessionStorage	~5 MB	a aba fechar	rascunhos e estado temporário
IndexedDB	centenas de MB	ser apagado	grandes volumes estruturados, offline pesado
Cache API	grande (cota do site)	ser apagado	respostas HTTP completas, Service Workers

Atenção de segurança que vale para todos: qualquer script da página consegue ler esses armazenamentos. Nunca guarde senhas, tokens sensíveis ou dados de cartão neles. Cache é para dados públicos e preferências, não para segredos.

Antes do cache, uma refatoração: a camada de serviços

Quando implementamos o CEP no capítulo anterior, a função de busca morava dentro do próprio componente da página. Funcionava, mas cada página nova que consumisse API copiaria a mesma estrutura de fetch, try/catch e json(). Repetição é convite para bug.

A solução foi criar a pasta `src/js/components/services` e centralizar a busca em um único arquivo:

```
// services/api.js
async function buscarServicos(url, dados='', forma='') {
  try {
    const formataURL = `${url}${dados}${forma}`
    const response = await fetch(formataURL);
    const result = await response.json();
    return result
  } catch (error) {
    console.error(error);
  }
};

export default buscarServicos;
```

Uma função genérica que monta a URL em três partes (base, dado variável e sufixo) e serve qualquer API. A tela de cadastro agora chama:

```
const dados = await buscarServicos("https://viacep.com.br/ws/", event.target.value, "/json/")
```

E a página de personagens chama a mesma função para outra API:

```
const detalhes = await buscarServicos("https://rickandmortyapi.com/api/character/", nPagina);
```

Guarde esse movimento: concentrar toda a conversa com a rede em um ponto único. Ele parece só organização, mas é o que vai tornar o cache quase de graça daqui a pouco.

O padrão Strategy: estratégias de armazenamento intercambiáveis

Antes de escrever o cache, uma decisão de projeto: onde guardar as cópias? Em memória (um Map, rápido, mas morre no F5)? No localStorage (sobrevive ao F5)? Amanhã, no IndexedDB?

Em vez de amarrar o cache a uma resposta, o projeto define um contrato: qualquer armazenamento serve, desde que saiba responder `has`, `get` e `set`. Cada implementação desse contrato é uma estratégia, e o arquivo `services/storageStrategy.js` traz duas:

```
// services/storageStrategy.js
const Memoria = {
  _cache: new Map(),

  has(key) {
    return this._cache.has(key);
  },
  get(key) {
    return this._cache.get(key);
  },
  set(key, value) {
    this._cache.set(key, value);
  }
};

const LocalStorage = {
  has(key) {
    return localStorage.getItem(key) !== null;
  },
  get(key) {
    const data = localStorage.getItem(key);
    return data ? JSON.parse(data) : null;
  },
  set(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
  }
};

export { Memoria, LocalStorage };
```

Repare que a estratégia `LocalStorage` esconde dentro de si o detalhe do `JSON.stringify` e do `JSON.parse`. Quem usar a estratégia não precisa saber que o `localStorage` só aceita texto; o contrato entrega e recebe objetos prontos.

Isso é um padrão de projeto clássico chamado Strategy: uma família de algoritmos intercambiáveis atrás de uma mesma interface. Trocar de estratégia é trocar uma palavra, como veremos já já. É o mesmo espírito do contrato das nossas páginas (url, label, pagina): quem cumpre o contrato entra no jogo sem que o resto do código mude.

O decorator: um cache que envolve a busca

Agora a peça central. O arquivo `services/apiCache.js` cria uma função que envolve a `buscarServicos` original, adicionando o comportamento de cache por fora, sem alterar uma linha dela:

```
// services/apiCache.js
import buscarServicos from "../api.js";
import { Memoria, LocalStorage } from "../storageStrategy.js";

const storage = LocalStorage;

async function buscarComCache(url, dados = '', forma = '') {
  const formataURL = `${url}${dados}${forma}`;
  if (storage.has(formataURL)) {
    console.time(`[CACHE] Tempo para: ${dados} || 'página inicial'`);
    const resultadoEmCache = storage.get(formataURL);
    console.timeEnd(`[CACHE] Tempo para: ${dados} || 'página inicial'`);
    return resultadoEmCache;
  }
  console.time(`[SERVIDOR] Tempo para: ${dados} || 'página inicial'`);
  try {
    const resultadoServidor = await buscarServicos(url, dados, forma);
    storage.set(formataURL, resultadoServidor);
    console.timeEnd(`[SERVIDOR] Tempo para: ${dados} || 'página inicial'`);
    return resultadoServidor;
  } catch (error) {
    console.timeEnd(`[SERVIDOR] Tempo para: ${dados} || 'página inicial'`);
    console.error("Erro na busca:", error);
    throw error;
  }
}

export default buscarComCache;
```

Vamos apreciar as decisões, uma a uma:

A URL completa é a chave do cache. Simples e genial: `...character/?page=1` e `...character/?page=2` geram entradas separadas automaticamente. Cada recurso único tem seu próprio cache sem nenhum esforço extra.

Cache primeiro. Se a estratégia tem a chave, devolvemos a cópia local e a rede nem é acordada. Se não tem, buscamos pela função original e guardamos o resultado para a próxima.

A estratégia é plugável. A linha `const storage = LocalStorage` é o interruptor: troque por `Memoria` e todo o comportamento de persistência muda, sem tocar em mais nada. O Strategy do arquivo anterior em ação.

A prova está no cronômetro. Os `console.time` e `console.timeEnd` medem e imprimem no console o tempo de cada caminho. Abra o console e navegue: a busca no servidor leva dezenas ou centenas de milissegundos; a resposta do cache, frações de milissegundo. Não acredite em mim, leia os seus próprios números.

Esse desenho, uma função que envolve outra preservando a mesma assinatura e adicionando comportamento, é conhecido como decorator (e, quando o objetivo é controlar o acesso ao objeto original, proxy). Repare no detalhe que sustenta tudo: `buscarComCache(url, dados, forma)` recebe exatamente os mesmos parâmetros de `buscarServicos(url, dados, forma)`. Assinaturas iguais tornam as duas intercambiáveis.

A troca de uma linha

E aqui a recompensa da refatoração para a camada de serviços. Para a tela de cadastro e a página de personagens ganharem cache, a mudança foi somente o import:

```
// antes
import buscarServicos from "../services/api.js"

// depois
import buscarServicos from "../services/apiCache.js"
```

Nada mais mudou nas páginas: como o decorator tem a mesma assinatura da função original, o resto do código nem percebe a troca. Se o fetch estivesse espalhado por cada componente, essa melhoria exigiria caçar e editar todos eles. Boas decisões de estrutura tornam melhorias futuras baratas; essa lição vale mais que o próprio cache.

Faça o teste de medição do início do capítulo de novo: página 1, página 2, volta para a 1. Agora a aba Network fica quieta na revisita e o console imprime o tempo do caminho `[CACHE]`.

As estratégias de cache clássicas

O que implementamos tem nome no mercado: cache first, olhe o cache antes, rede só na falta. Existe um vocabulário de estratégias, e cada uma resolve um compromisso diferente entre velocidade e frescor dos dados:

- Cache first: responde do cache; rede apenas quando não há cópia. Máxima velocidade, ideal para dados que mudam pouco. É a nossa
- Network first: tenta a rede; se falhar (offline, servidor fora), cai para o cache. Ideal para dados que precisam estar atualizados, com o cache como plano B
- Stale while revalidate: responde o cache imediatamente e, em paralelo, busca a versão nova para a próxima visita. O usuário nunca espera e o dado nunca fica muito velho. É o queridinho de feeds e notícias
- Network only e cache only: os extremos, sem cache e somente cache, para casos especiais

E a peça que falta na nossa: validade. Nosso cache é eterno; se a API mudar, mostraremos a cópia velha para sempre (eis a invalidação de cache da frase do Karlton). A técnica clássica é o TTL, time to

live: guardar junto com o dado o momento em que foi salvo e, na leitura, conferir a idade:

```
// evolucao da estrategia LocalStorage com validade de 10 minutos
const VALIDADE_MS = 10 * 60 * 1000;

set(key, value) {
  localStorage.setItem(key, JSON.stringify({ dados: value, salvoEm: Date.now() }));
},
get(key) {
  const bruto = localStorage.getItem(key);
  if (!bruto) return null;
  const { dados, salvoEm } = JSON.parse(bruto);
  if (Date.now() - salvoEm > VALIDADE_MS) return null; // expirou
  return dados;
}
```

Repare onde a mudança mora: dentro da estratégia, não no decorator. O apiCache continua intocado. Cada peça da arquitetura absorve a evolução que lhe diz respeito.

A estratégia que falta no projeto: o Service Worker

Todas as estratégias acima rodam dentro da página. Existe um nível acima, que ainda não está no nosso código e fica aqui como fronteira a explorar: o Service Worker.

Um Service Worker é um script que o navegador roda em segundo plano, separado da página, e que funciona como um porteiro da rede: toda requisição do site passa por ele, incluindo o HTML, o CSS, as imagens e os scripts, não só as chamadas de API:

```
sem service worker:
  pagina — fetch → internet

com service worker:
  pagina — fetch → [ service worker ] → internet
                        |
                        → Cache API (pode responder sem rede)
```

Com ele, as estratégias de cache saem do nosso decorator e passam a valer para a aplicação inteira, o que permite o feito máximo: o site abrir sem internet nenhuma. É a tecnologia por trás dos PWAs, os aplicativos instaláveis feitos com tecnologia web. Um esqueleto para reconhecer as peças:

```
// sw.js: o porteiro da rede
const CACHE = "app-livros-v1";

self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(CACHE).then((cache) =>
      cache.addAll(["/", "/index.html", "/src/css/microframework.css",
"/src/js/main.js"])
    )
  );
});

self.addEventListener("fetch", (event) => {
  // estrategia cache first para tudo
  event.respondWith(
    caches.match(event.request).then((doCache) => doCache || fetch(event.request))
  );
});

// e na pagina, o registro:
// navigator.serviceWorker.register("/sw.js");
```

Você reconhece tudo: `addEventListener` do Capítulo 7, Promises do Capítulo 10, a Cache API deste capítulo e a estratégia `cache first` que acabamos de implementar à mão. O Service Worker exige HTTPS (ou localhost) justamente por ser poderoso demais, e tem um ciclo de vida próprio (`install`, `activate`, `fetch`) que merece um estudo dedicado. Fica como o desafio de fronteira do curso: os fundamentos você já tem todos.

Resumo do capítulo

- Cache é uma cópia local de um dado caro de buscar; a web inteira funciona sobre camadas de cache
- O navegador oferece cinco armazenamentos: cookies (sessão com o servidor), `localStorage` (persistente), `sessionStorage` (por aba), `IndexedDB` (banco local) e Cache API (respostas HTTP)
- O `localStorage` só guarda texto; `JSON.stringify` e `JSON.parse` fazem a ponte
- A camada de serviços (`services/api.js`) centralizou a rede e tornou o cache uma troca de import
- O `storageStrategy.js` aplica o padrão Strategy: Memória e `LocalStorage` cumprem o mesmo contrato `has`, `get` e `set`
- O `apiCache.js` é um decorator/proxy: envolve a busca original com a lógica de cache, mantendo a mesma assinatura, e prova o ganho com `console.time`
- Estratégias clássicas: `cache first` (a nossa), `network first`, `stale while revalidate`, e `TTL` para invalidar cache velho
- O Service Worker leva as estratégias para toda a aplicação e habilita o offline completo; é o próximo degrau

Para praticar

1. Navegue pelas páginas de personagens com o console aberto e anote os tempos de `[SERVIDOR]` e `[CACHE]`. Calcule quantas vezes o cache é mais rápido.
2. Troque a estratégia para `Memoria` no `apiCache`, recarregue com F5 e explique a diferença de comportamento em relação à `LocalStorage`.
3. Implemente o TTL dentro da estratégia `LocalStorage` com validade de 30 segundos e observe no console o cache expirar.
4. Crie uma terceira estratégia, `SessionStorage`, cumprindo o mesmo contrato, e ligue-a no `apiCache`.
5. Escreva uma função `limparCache()` que remove do `localStorage` apenas as chaves que começam com `http`, preservando outras preferências. Dica: percorra `Object.keys(localStorage)`.
6. Desafio: aplique o esqueleto de Service Worker ao projeto rodando no Live Server, e confirme na aba Application do DevTools que ele foi registrado. Depois desligue a rede e recarregue.
7. Reflexão: para um site de notícias, qual estratégia você escolheria, cache first, network first ou stale while revalidate? Justifique pelo compromisso entre velocidade e frescor.

Referências

- MDN Web Docs, Web Storage API: https://developer.mozilla.org/pt-BR/docs/Web/API/Web_Storage_API
- MDN Web Docs, IndexedDB API: https://developer.mozilla.org/pt-BR/docs/Web/API/IndexedDB_API
- MDN Web Docs, Cache API: <https://developer.mozilla.org/pt-BR/docs/Web/API/Cache>
- MDN Web Docs, Service Worker API: https://developer.mozilla.org/pt-BR/docs/Web/API/Service_Worker_API
- MDN Web Docs, HTTP caching: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Caching>
- web.dev (Google), Caching strategies e offline: <https://web.dev/learn/pwa/caching>
- W3Schools, Web Storage: https://www.w3schools.com/html/html5_webstorage.asp
- Refactoring Guru, padrões Strategy, Decorator e Proxy: <https://refactoring.guru/pt-br/design-patterns>

Capítulo 13: Programando com IA, engenharia de prompt para desenvolvedores

Uma conversa franca antes de começar

Chegamos à penúltima aula com uma pergunta que paira sobre todo estudante de programação hoje: se a inteligência artificial escreve código, por que passei meses aprendendo variáveis, laços, funções, DOM, assincronismo e fetch?

A resposta é o tema deste capítulo: a IA escreve código, mas quem projeta, avalia, corrige e responde pelo resultado é você. Um LLM nas mãos de quem não domina os fundamentos produz código que a pessoa não sabe ler, não sabe testar e não sabe consertar quando quebra. Nas mãos de quem fez este curso, produz velocidade: você vira o arquiteto e o revisor, e a IA vira um programador funcionário da sua equipe, rápido e incansável, mas que precisa de ordens claras e de supervisão.

Repare no que aconteceu nos capítulos anteriores: você sabe por que o `preventDefault` é obrigatório no submit, por que o listener vem depois do `innerHTML`, por que o fetch precisa de dois awaits, por que o mapa de rotas vence o switch. É exatamente esse conhecimento que transforma um prompt vago em uma especificação precisa, e uma resposta da IA em código auditado. A estrutura é sua; a digitação é dela.

O que é um LLM

LLM significa Large Language Model, grande modelo de linguagem. São sistemas treinados sobre volumes gigantescos de texto e código (incluindo a documentação do MDN e milhões de repositórios) que aprendem a prever a continuação mais provável de um texto. Quando você escreve um pedido, o modelo gera a resposta token a token com base nesses padrões. Exemplos: ChatGPT, Claude, Gemini e Copilot.

Duas consequências práticas desse funcionamento:

1. O modelo não executa o código que escreve. Ele prevê texto plausível. Código plausível e código correto são coisas diferentes, e a diferença aparece quando você testa
2. O modelo pode alucinar: afirmar com confiança algo falso, como um método que não existe ou um parâmetro inventado. A defesa é a sua: testar no navegador e conferir no MDN

Por isso a regra de ouro do capítulo: a IA propõe, você dispõe. Nenhuma linha entra no projeto sem você entender o que ela faz.

Anatomia de um bom prompt

Prompt é a instrução que você dá ao modelo. A diferença entre uma resposta genérica e uma resposta sob medida está quase toda na qualidade do prompt. Um bom prompt de programação tem estas partes:

1. Papel (persona): quem a IA deve ser. Define o tom, o nível e as prioridades da resposta

2. Contexto: o projeto, as tecnologias, as restrições e os padrões existentes
3. Tarefa: o que exatamente deve ser feito, com critérios de aceitação
4. Formato da saída: como você quer receber (arquivo completo, apenas o trecho, com ou sem explicação)
5. Restrições: o que não pode ser feito

Compare um prompt fraco com um forte, para a mesma necessidade real do nosso projeto:

Prompt fraco:

```
faz uma pagina de livros em javascript
```

O resultado será qualquer coisa: talvez React, talvez jQuery, talvez um HTML solto que não conversa com nossa arquitetura.

Prompt forte:

```
Você é um desenvolvedor front-end da minha equipe, especialista em JavaScript puro (Vanilla JS), sem frameworks.
```

```
Contexto do projeto: uma SPA com roteamento por hash. Cada página é um componente auto montável em um arquivo próprio, seguindo este contrato: uma função async que recebe o elemento "app" e se desenha nele via innerHTML, registrando os próprios eventos depois de desenhar; e um export default de um objeto { url, label, pagina }. Exemplo real do projeto:
```

```
async function telaCadastro(app){
  const formulario = `...html...`
  app.innerHTML = formulario;
  await capturacep()
}
export default {
  url: '#cep',
  label: 'Cadastro',
  pagina: telaCadastro
}
```

```
O CSS usa metodologia BEM com classes como bem-card, bem-card__title e bem-grid-auto.
```

```
Tarefa: crie o componente de uma página "Livros" que busca dados da API https://openlibrary.org/search.json?q=javascript com fetch e async/await, trata erros com try/catch e testa response.ok, e renderiza os 10 primeiros resultados como cards (título e autor) usando as classes BEM citadas.
```

```
Formato: entregue apenas o arquivo livros.js completo, com comentários curtos explicando os pontos principais.
```

```
Restrições: não use frameworks nem bibliotecas externas, não altere nenhum outro arquivo, e não use var.
```

Percebeu o que aconteceu? Todo o conhecimento do curso virou especificação: o contrato dos componentes (Capítulo 8), o padrão de fetch com tratamento de erro (Capítulos 10 e 11), o BEM

(Capítulo 8), a proibição do var (Capítulo 2). A IA que receber esse prompt vai produzir código que encaixa no projeto na primeira tentativa. Quem não fez o curso não tem como escrever esse prompt.

Prompts de sistema: criando seu programador funcionário

Ferramentas de IA permitem definir uma instrução permanente (system prompt, instruções personalizadas ou arquivos de contexto do projeto, como um CLAUDE.md). É ali que você cria de vez a personalidade e as regras do seu assistente, sem repetir tudo a cada pergunta. Um modelo pronto para o nosso projeto, que você pode adaptar:

```
# Papel
Você é um desenvolvedor front-end júnior da minha equipe. Eu sou o
desenvolvedor responsável pelo projeto e reviso todo o seu código.

# Projeto
SPA de uma livraria em JavaScript puro (ES2015+), sem frameworks.
Roteamento por hash: um array de rotas vira um mapa (objeto) indexado
pela url; o evento hashchange dispara a função render, que executa a
função "pagina" da rota atual passando o elemento #app. Rotas
desconhecidas caem em uma rota 404.

# Padrões obrigatórios
- Cada página é um componente auto montável: função async que recebe o
  app, desenha via innerHTML e depois registra os próprios listeners
- Export default de objeto { url, label, pagina }
- const por padrão, let quando necessário, var proibido
- Template strings para todo HTML gerado
- Eventos com addEventListener, nunca atributos onclick no HTML
- Fetch sempre com async/await, try/catch e teste de response.ok
- Conteúdo digitado por usuário entra no DOM via textContent, nunca innerHTML
- CSS com metodologia BEM

# Como responder
- Antes de codificar, liste em uma frase o que entendeu da tarefa
- Entregue arquivos completos, indicando o caminho de cada um
- Explique decisões não óbvias em no máximo três linhas
- Se a tarefa estiver ambígua, faça perguntas antes de codificar
- Se algo que pedi violar os padrões acima, avise antes de fazer

# Limites
- Não altere arquivos que não fazem parte da tarefa
- Não adicione bibliotecas nem frameworks
- Não invente APIs: se não tiver certeza de um método, diga que é
  preciso confirmar no MDN
```

Esse texto é um documento de arquitetura disfarçado de prompt. Escrivê-lo exige dominar o projeto; mantê-lo atualizado é parte do trabalho de quem lidera.

Técnicas que elevam a qualidade das respostas

Além da estrutura, algumas técnicas comprovadas:

Dê exemplos (few-shot). Mostrar um componente pronto, como fizemos no prompt forte, vale mais que três parágrafos de descrição. O modelo imita padrões com muita eficiência.

Peça o raciocínio antes do código. Instruções como "explique sua abordagem em três passos antes de codificar" reduzem erros de lógica, porque forçam o modelo a planejar.

Divida tarefas grandes. Não peça "faça o sistema de livros completo". Peça o componente da lista, depois a busca, depois a página de detalhe. Igual dividimos o curso em aulas, divida o trabalho em prompts.

Itere sobre a resposta. A primeira resposta é um rascunho. Continue a conversa: "o listener está sendo registrado antes do innerHTML, corrija", "extraia a montagem do card para uma função separada". Saber apontar o erro exato é o seu diferencial.

Use a IA para aprender, não só para produzir. Alguns dos melhores prompts de estudante:

Explique linha por linha o que este código faz, como se eu fosse um aluno que acabou de aprender eventos: [cole o código]

Este código quebra com o erro [cole o erro do console]. Explique a causa provável antes de propor a correção.

Revise este meu componente apontando problemas de segurança, de legibilidade e violações do contrato do projeto: [cole o código]

Me faça cinco perguntas de prova sobre assincronismo em JavaScript e corrija minhas respostas uma a uma.

O fluxo de trabalho profissional com IA

Na prática de mercado, o ciclo com um assistente de IA é este, e repare quem faz cada parte:

1. Você especifica: papel, contexto, tarefa, formato, restrições
2. A IA produz um rascunho
3. Você lê e entende cada linha. O que não entender, pergunta antes de usar
4. Você testa no navegador, com o console e a aba Network abertos
5. Você refina em novas rodadas de conversa
6. Você faz o commit, porque a responsabilidade pelo código é de quem assina

A IA participa apenas do passo 2. Todos os outros dependem dos fundamentos que você construiu nesta apostila. É por isso que os melhores usuários de IA são justamente os que menos precisariam dela: conhecimento técnico não foi substituído, foi promovido a cargo de chefia.

Cuidados éticos e de segurança

- Nunca cole em ferramentas de IA senhas, chaves de API, tokens ou dados pessoais de clientes

- Código gerado pode reproduzir padrões inseguros; a revisão de segurança (como o cuidado com innerHTML e XSS do Capítulo 7) continua sendo sua
- Em processos seletivos, provas e trabalhos, respeite as regras sobre uso de IA; e lembre que entrevistas técnicas testam você sem assistente
- Cite e confira as fontes: quando a IA afirmar algo sobre a linguagem, valide no MDN

Resumo do capítulo

- LLMs preveem texto plausível; correção é responsabilidade de quem revisa e testa
- Um bom prompt tem papel, contexto, tarefa, formato e restrições
- O prompt de sistema transforma a IA em um programador funcionário com a personalidade e os padrões que você definir
- Exemplos, raciocínio antes do código, divisão de tarefas e iteração elevam a qualidade
- O fluxo profissional: você especifica, a IA rascunha, você entende, testa, refina e assina
- Os fundamentos do curso são o que permitem escrever boas especificações e auditar os resultados

Para praticar

1. Escreva um prompt fraco e um forte para criar a página "Autores" do projeto, e compare as respostas de uma IA para os dois.
2. Adapte o prompt de sistema deste capítulo para o seu projeto pessoal e salve como documento do projeto.
3. Peça a uma IA que gere um componente novo e depois encontre você mesmo, sem ajuda, três pontos a melhorar na resposta.
4. Cole um trecho da apostila que ainda gere dúvida e peça uma explicação com analogias diferentes.
5. Desafio final: usando seu prompt de sistema, conduza uma IA na criação de uma funcionalidade completa para o App Livros em pelo menos três rodadas de refinamento, e apresente na aula o histórico da conversa comentando suas intervenções.

Referências

- MDN Web Docs (fonte para validar qualquer afirmação técnica): <https://developer.mozilla.org/pt-BR/>
- Anthropic, guia de engenharia de prompt: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- OpenAI, boas práticas de prompt: <https://platform.openai.com/docs/guides/prompt-engineering>
- Google, introdução aos LLMs: <https://developers.google.com/machine-learning/resources/intro-llms>
- W3Schools, seção de IA generativa: https://www.w3schools.com/gen_ai/

Capítulo 14: Fechando o ciclo, o projeto completo e os próximos passos

Olhando para trás

Na primeira aula, nossa aplicação era um `console.log` no navegador. Hoje ela é uma SPA com menu dinâmico, cinco páginas componentizadas, roteamento por hash com página 404 e um formulário que consulta uma API pública em tempo real. Nenhum framework, nenhuma mágica: cada linha passou pela sua mão e está explicada nesta apostila.

Vale reconstituir o mapa do que foi aprendido, porque ele é também o mapa de leitura para revisão:

Fundamento	Onde vive no projeto	Capítulo		
Variáveis, const e let, template strings	Todo o projeto, em especial as páginas	2		
Condicionais, operador \	\	como padrão	Hash inicial e rota 404 no main.js	3
Laços for e for...of	Cards de serviços e mapa de rotas	4		
Arrays e objetos, .map()	rotas.js, detalhes de serviços, navbar	5		
Funções, callbacks, arrow functions	Contrato das páginas, rota 404, listeners	6		
DOM e eventos (submit, blur, hashchange)	Contato, cadastro e roteador	7		
ES Modules e componentes auto montáveis	Estrutura inteira do src/js	8		
Roteamento hash em SPA	main.js linha por linha	9		
Event loop, Promises, async/await	Testes comentados e render assíncrono	10		
Fetch, JSON e tratamento de erros	Cadastro com ViaCEP e personagens	11		
Cache, armazenamentos e padrões Strategy e Decorator	services/api.js, apiCache.js e storageStrategy.js	12		
Engenharia de prompt e trabalho com IA	Seu próximo projeto	13		

A prova de que você aprendeu

O teste definitivo não é recitar definições, é conseguir responder perguntas de arquitetura como estas, que foram as perguntas centrais do curso:

1. Por que a aplicação inteira funciona com um único arquivo HTML?
2. O que acontece, passo a passo, entre o clique em "Sobre" no menu e a página aparecer na tela?
3. Por que adicionar uma página nova não exige alterar o roteador nem a navbar?
4. Por que o `addEventListener` precisa vir depois do `innerHTML` ?
5. Por que o `fetch` precisa de `await` duas vezes e de um `try/catch` ?

Se alguma resposta hesitar, o capítulo correspondente está aí para revisita. A apostila foi escrita para ser relida com o projeto aberto do lado.

Próximos passos do projeto

O roadmap do curso ainda reserva refinamentos, e eles são os desafios de conclusão:

1. Renderização dinâmica de dados de API: substituir o array `detalhes` da página de Serviços por dados reais vindos de uma API de livros, como a Open Library, aplicando o padrão completo do Capítulo 11
2. Persistência local: guardar os contatos capturados no `localStorage` , para a lista sobreviver ao fechamento do navegador
3. Validações de formulário: CEP com formato correto, email válido, campos obrigatórios
4. Estados de carregamento: mostrar um "carregando..." enquanto o `fetch` não responde
5. Acessibilidade e refinamento de CSS sobre a base BEM já construída

E o desafio maior, que usa o Capítulo 13: escolher uma funcionalidade dessa lista e construí-la conduzindo uma IA como seu programador funcionário, com você no papel de arquiteto e revisor.

Depois do curso

O caminho natural a partir daqui:

- Aprofundar JavaScript: classes, closures, destructuring, spread, generators
- Aprender um framework (React ou Vue) reconhecendo neles tudo que você já fez na mão: componentes, rotas, renderização orientada a dados
- Conhecer testes automatizados, que são o próximo nível da confiança no próprio código
- Praticar Git além do básico: branches, pull requests, revisão de código

Bibliografia e referências gerais

Documentação de consulta permanente

- MDN Web Docs, a referência definitiva de JavaScript, HTML e CSS, mantida pela Mozilla: <https://developer.mozilla.org/pt-BR/>

- W3Schools, tutoriais diretos com editor de testes online, bom para consulta rápida: <https://www.w3schools.com/js/>
- Especificação viva do JavaScript (ECMAScript), para os curiosos: <https://tc39.es/ecma262/>

Livros da Casa do Código

- SILVEIRA, Paulo; ALMEIDA, Adriano. Lógica de Programação: crie seus primeiros programas usando Javascript e HTML. Casa do Código. Ideal para revisar os fundamentos dos primeiros capítulos com outra abordagem
- ALMEIDA, Flávio. Cangaceiro JavaScript: uma aventura no sertão da programação. Casa do Código. Aprofunda JavaScript avançado, padrões de projeto e SPA sem framework, exatamente o espírito do nosso curso
- TEIXEIRA, Stefan. JavaScript Assertivo: testes e qualidade de código em JS de ponta a ponta. Casa do Código. O próximo passo natural: garantir com testes que o código continua funcionando

Catálogo completo da editora: <https://www.casadocodigo.com.br/>

Outros livros recomendados

- HAVERBEKE, Marijn. Eloquent JavaScript. Disponível gratuitamente em <https://eloquentjavascript.net/> (em inglês, com tradução parcial em português)
- SIMPSON, Kyle. Série You Don't Know JS Yet. Disponível gratuitamente em <https://github.com/getify/You-Dont-Know-JS> (em inglês), referência profunda sobre escopo, closures e assincronismo

Ferramentas usadas no curso

- Visual Studio Code: <https://code.visualstudio.com/>
- Extensão Live Server para servir a SPA localmente
- Git e GitHub para versionamento, com o histórico do projeto servindo de linha do tempo das aulas
- DevTools do navegador (F12): Console para depuração e Network para inspecionar as requisições

Palavra final

Este projeto foi construído para ser modelo: a estrutura de pastas, o contrato dos componentes, o roteador e os padrões de código estão prontos para você copiar, adaptar e expandir na sua própria aplicação. O código está no repositório, a explicação está nesta apostila e o método de trabalho com IA está no Capítulo 13.

A partir de agora, o professor vira consultor e o aluno vira autor. Bom código.